Lab Lecture

REST API with Python

Dr. Ei Ei Thu
Associate Professor
Faculty of Computer Science

2024-2025 Academic Year

Outlines

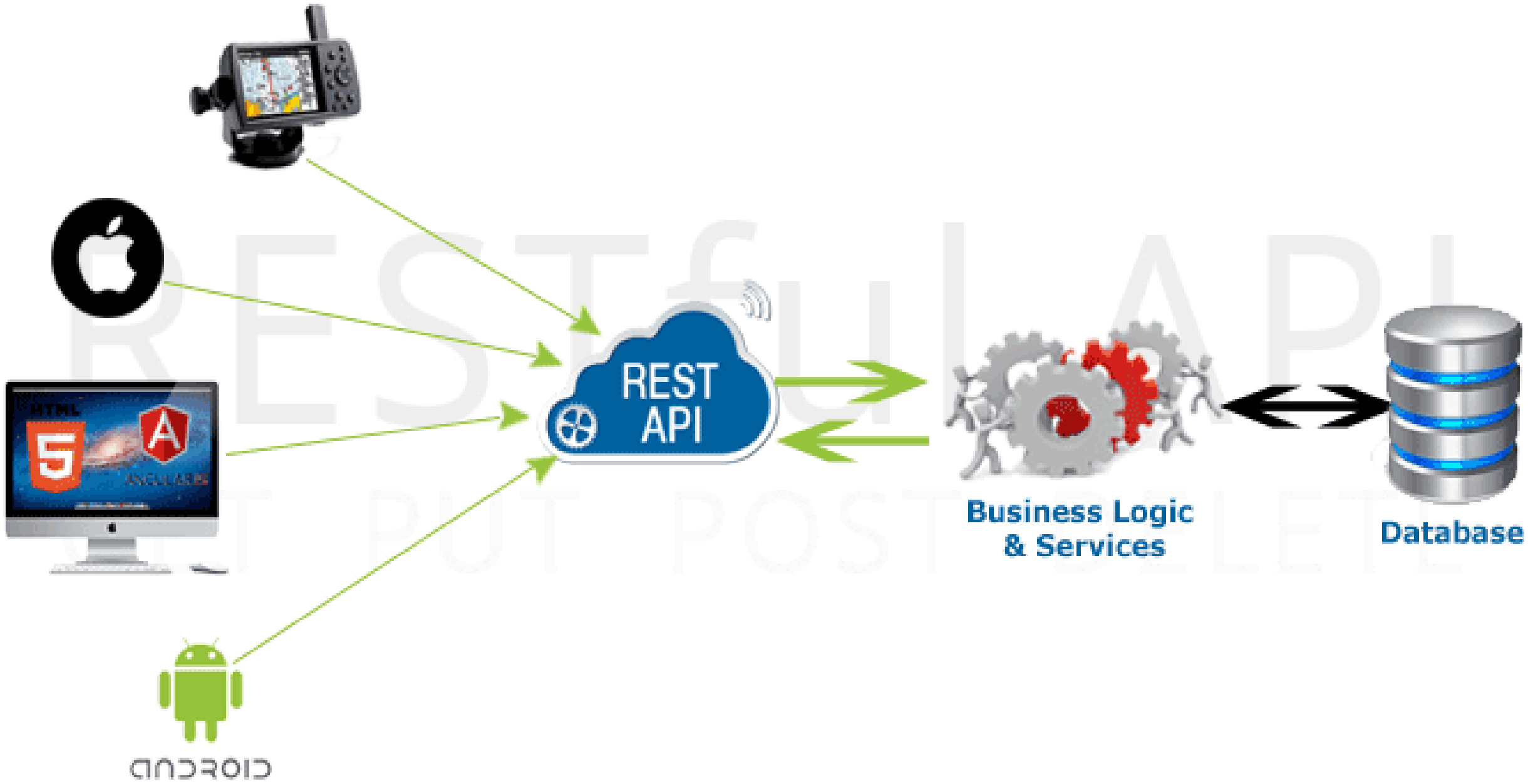
Introduction

Python Flask Example

Distributed System Example Scenarios

Fetch Real World API

Fetch Machine Learning Model via API

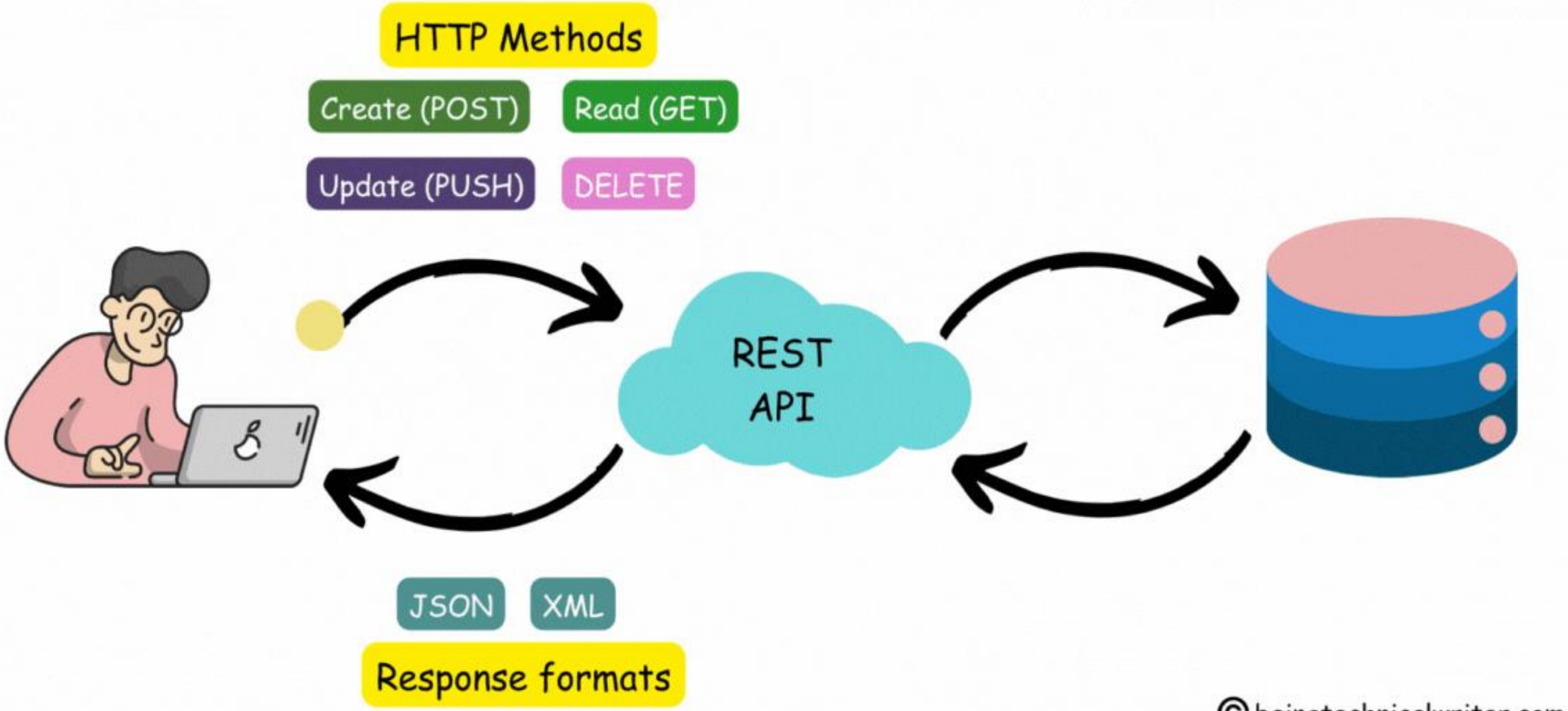


Introduction

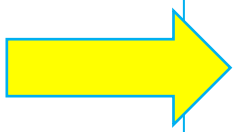
REST stands for **RE**presentational **S**tate **T**ransfer.

API stands for **A**pplication **P**rogramming **I**nterface.

REST API allows applications to interact with each other and exchange data.


HTTP Methods
Create (POST)
Read (GET)
Update (PUSH)
DELETE
REST API
JSON
XML
Response formats
© beingtechnicalwriter.com

This lecture will cover only GET operation



HTTP Verbs	CRUD Operations
Post	Create
Get	Read
Put or Patch	Update
Delete	Delete

URI	HTTP Verb	Outcome
.../api/employees	GET	Gets list of employees
.../api/employees/1	GET	Gets employee with Id = 1
.../api/employees	POST	Creates a new employee
.../api/employees/1	PUT	Updates employee with Id = 1
.../api/employees/1	DELETE	Deletes employee with Id = 1

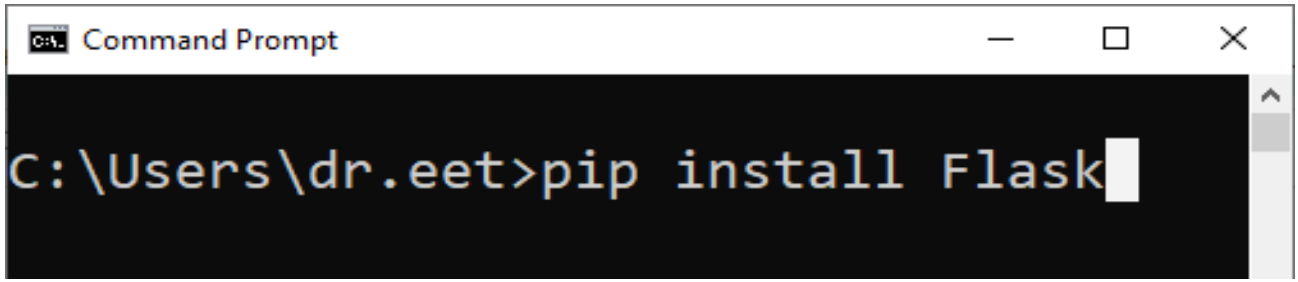
REST API using Flask

Python Flask Example

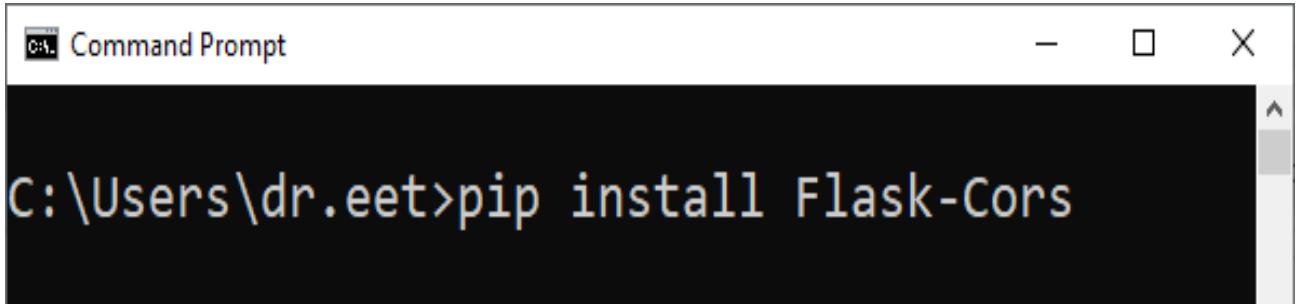
Flask is a lightweight and powerful web framework (micro-framework) for Python.

Flash functions on the principle of a Web Server Gateway Interface (WSGI).

Flask allows Python developers to create lightweight REST APIs.



Flask-CORS is a Flask extension that enables Cross-Origin Resource Sharing support for Flask applications.



Python Flask Example

```
from flask import Flask, request, render_template_string

app=Flask(__name__)

# HTML template
html_template = '''
<!DOCTYPE html>
<html>
<head><title>Simple Flask App</title></head>
<body>
<h1>Flask Test</h1>
  <h2>Enter Your Name</h2>
  <form method="POST">
    <input type="text" name="username">
    <input type="submit" value="Submit">
  </form>

  {% if name %}
    <h3>Hello, {{ name }}!</h3>
  {% endif %}
</body>
</html>
'''
```

Python Flask Example

```
@app.route('/', methods=['GET', 'POST'])
def home():
    name=None
    if request.method=='POST':
        name=request.form.get('username')
    return render_template_string(html_template, name=name)

def run_app():
    app.run("172.21.13.167",port=5000)

#import threading
#thread=threading.Thread(target=run_app)
#thread.start()
```

Example Scenarios for Simple Distributed System

1. **Client-Server Architecture in Distributed System**

   Python API + JavaScript frontend (browser on another server - Tomcat)

   Python Flask serves /api/students.

   JavaScript (React/HTML on another server) fetches that API.

2. **Multi-service Distributed System**

   Python provides JSON data.

   PHP fetches that API and renders HTML.

   two backend servers working together

3. **Microservices-based Distributed System (IPC)**

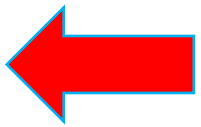
   Microservice A (KPay) manages user wallet balances

   Microservice B (Atom Telecom) manages telecom account balances

   Microservice C (Consumer) coordinates the workflow of payments

1. Client-Server Architecture in Distributed System (Python API + JS on Tomcat)

```python
from flask import Flask, jsonify
from flask_cors import CORS  # To allow cross-origin requests

app = Flask(__name__)
CORS(app)
# Enable CORS so other servers (like your HTML frontend) can access it
# Cross-Origin Resource Sharing (CORS) extension or middleware

#student data
students = [
    {"id": 1, "name": "Rose", "age": 20, "grade": "A"},
    {"id": 2, "name": "Jiso", "age": 22, "grade": "B"},
    {"id": 3, "name": "Lisa", "age": 21, "grade": "A-"},
]

#function index()
@app.route("/")
def index():
    return jsonify(students)

# API endpoint
@app.route('/api/students', methods=['GET'])
def get_students():
    return jsonify(students)

from werkzeug.serving import run_simple
run_simple("172.21.13.167", 8282, app, use_reloader=False)
```



1. Client-Server Architecture in Distributed System (Python API + JS on Tomcat)

index.html

```html
<!DOCTYPE html>
<html>
<head>
  <title>Student Data</title>
</head>
<body>
  <h1>Student List</h1>
  <ul id="student-list"></ul>

  <script>
    // Change URL to your Python server IP/domain
    fetch("http://172.21.13.167:8282/api/students")
      .then(response => response.json())
      .then(data => {
        const list = document.getElementById("student-list");
        data.forEach(student => {
          const item = document.createElement("li");
          item.textContent = `${student.name} (Age: ${student.age}, Grade: ${student.grade})`;
          list.appendChild(item);
        });
      })
      .catch(error => console.error("Error fetching student data:", error));
  </script>
</body>
</html>
```

Create Dynamic Web Project with Eclipse IDE and Tomcat Server

2. Multi-service Distributed System (Python API + PHP on Apache)

index.php

index.php > html

```php
<?php
// API endpoint (adjust host/IP as needed)
$api_url = "http://172.21.13.167:8282/api/students";

// Fetch JSON data from Python API
$json_data = file_get_contents($api_url);

// Decode JSON into PHP array
$students = json_decode($json_data, true);
?>
<!DOCTYPE html>
<html>
<head>
    <title>Student Data from Python API</title>
</head>
<body>
    <h1>Student List</h1>
    <ul>
        <?php foreach ($students as $student): ?>
            <li>
                <?php echo htmlspecialchars($student['name']); ?>
                (Age: <?php echo htmlspecialchars($student['age']); ?>,
                Grade: <?php echo htmlspecialchars($student['grade']); ?>)
            </li>
        <?php endforeach; ?>
    </ul>
</body>
</html>
```

Create PHP Web Project with VSCode and Apache Server

3. Microservices Architecture and Interprocess Communication in a Distributed System

Microservice A (KPay) manages user wallet balances

Microservice B (Atom Telecom) manages telecom account balances

Microservice C (Consumer) coordinates the workflow of payments

Each service has:

its own data (separate database)

its own API

runs independently on different ports (7001, 7002, 7003)

Interprocess Communication (IPC): Services are separate processes and can run on different servers.

User – HTML in Service C - /pay-bill
Service C (Consumer) - /pay
Service A (KPay)
- Deducts KPay balance
- Returns new KPay balance → Service C
- Also calls Service B (/add)

Service B (Atom)
- Updates its balance
- Returns new Telecom balance → Service C

Service C
- Shows both balances (KPay + Telecom) to **User**

All data exchange is in **JSON** over HTTP.

Microservice A (KPay) Synchronous

```python
from flask import Flask, request, jsonify
import requests
from flask_cors import CORS

app = Flask(__name__)
CORS(app)

kpay_balances = {1: 20000, 2: 15000, 3: 10000}
TELECOM_URL = "http://172.21.13.167:7002"

@app.route('/pay', methods=['POST'])
def process_payment():
    data = request.get_json()
    user_id = data.get("user_id")
    amount = int(data.get("amount", 0))

    if user_id not in kpay_balances:
        return jsonify({"error": "User not found"}), 404

    if kpay_balances[user_id] < amount:
        return jsonify({"status": "failed", "message": "Insufficient balance"}), 400

    # Deduct balance
    kpay_balances[user_id] -= amount

    # Trigger telecom update (but Service B reports back to C, not to A)
    telecom_payload = {"user_id": user_id, "amount": amount}
    requests.post(f"{TELECOM_URL}/add", json=telecom_payload)

    # Return only KPay balance
    return jsonify({
        "status": "success",
        "message": f"Deducted {amount} from KPay for user {user_id}",
        "user_id": user_id,
        "kpay_balance": kpay_balances[user_id]
    })
from werkzeug.serving import run_simple
run_simple("172.21.13.167", 7001, app, use_reloader=False)
```

Microservice B (Atom) Synchronous

```python
from flask import Flask, request, jsonify
import requests
from flask_cors import CORS

app = Flask(__name__)
CORS(app)

telecom_balances = {1: 1000, 2: 3000, 3: 0}
CONSUMER_URL = "http://172.21.13.167:7003"

@app.route('/add', methods=['POST'])
def add_balance():
    data = request.get_json()
    user_id = data.get("user_id")
    amount = int(data.get("amount", 0))

    if user_id not in telecom_balances:
        return jsonify({"error": "User not found"}), 404

    # Add to telecom balance
    telecom_balances[user_id] += amount

    # Inform Service C (Consumer) about new balance
    notify_payload = {
        "user_id": user_id,
        "telecom_balance": telecom_balances[user_id]
    }
    try:
        requests.post(f"{CONSUMER_URL}/notify-telecom", json=notify_payload)
    except Exception as e:
        print("Failed to notify Consumer:", str(e))

    return jsonify({
        "status": "success",
        "user_id": user_id,
        "telecom_balance": telecom_balances[user_id]
    })

from werkzeug.serving import run_simple
run_simple("172.21.13.167", 7002, app, use_reloader=False)
```

Microservice C (User) Synchronous

```python
from flask import Flask, request, render_template_string
import requests
from flask_cors import CORS

app = Flask(__name__)
CORS(app)

KPAY_URL = "http://172.21.13.167:7001"

# Store balances in memory for demo
last_result = {}

HTML_PAGE = """
<!DOCTYPE html>
<html>
<head>
    <title>Pay Telecom Bill</title>
</head>
<body>
    <h1>Pay Atom Bill via KPay</h1>
    <form method="POST" action="/pay-bill">
        <label>User ID:</label>
        <input type="number" name="user_id" required><br><br>
        <label>Amount:</label>
        <input type="number" name="amount" required><br><br>
        <button type="submit">Pay Now</button>
    </form>

    {% if result %}
        <h2>Payment Result</h2>
        {% if result.get("message") %}<p>{{ result['message'] }}</p>{% endif %}
        {% if result.get("kpay_balance") %}<p><b>KPay Balance:</b> {{ result['kpay_balance'] }}</p>{%
        {% if result.get("telecom_balance") %}<p><b>Telecom Balance:</b> {{ result['telecom_balance']
    {% endif %}
</body>
</html>
"""

@app.route('/')
def home():
    return render_template_string(HTML_PAGE, result=last_result)
```

```python
@app.route('/')
def home():
    return render_template_string(HTML_PAGE, result=last_result)

@app.route('/pay-bill', methods=['POST'])
def pay_bill():
    import time
    user_id = int(request.form.get("user_id"))
    amount = int(request.form.get("amount"))

    # Call KPay
    payload = {"user_id": user_id, "amount": amount}
    resp = requests.post(f"{KPAY_URL}/pay", json=payload)
    last_result.update(resp.json())  # Update with KPay response

    # Short wait so Telecom can notify
    time.sleep(1)
    return render_template_string(HTML_PAGE, result=last_result)

@app.route('/notify-telecom', methods=['POST'])
def notify_telecom():
    data = request.get_json()
    last_result.update(data)  # Store telecom balance
    return {"status": "received"}

from werkzeug.serving import run_simple
run_simple("172.21.13.167", 7003, app, use_reloader=False)
```

Microservice C (User)
Synchronous

Synchronous Blocking Issues

1. Service C (/pay-bill) calls Service A /pay.
2. Service A /pay calls Service B /add synchronously.
3. Service B /add calls Service C /notify-telecom synchronously.
4. Meanwhile, Service C is still waiting for the response from Service A.

This creates a circular wait / blocking call:

- Service C waits for Service A →
- Service A waits for Service B →
- Service B waits for Service C →

Deadlock / request hangs.

Fix Issues with Asynchronous Non-Blocking

- Service B updates its internal balance.
- Instead of directly calling Service C, Service C polls or fetches the balance after payment, or uses a message queue / webhook.

Remove the circular call

1. Let Service C only call Service A.
2. Service A calls Service B.
3. Service A collects both KPay and Atom balances and returns them to Service C.
4. Service B no longer calls Service C directly.

This is simpler and avoids hanging.

Microservice A (KPay) Asynchronous

```python
from flask import Flask, request, jsonify
import requests
from flask_cors import CORS

app = Flask(__name__)
CORS(app)

kpay_balances = {1: 20000, 2: 15000, 3: 10000}
TELECOM_URL = "http://172.21.13.167:7002"

@app.route('/pay', methods=['POST'])
def process_payment():
    data = request.get_json()
    user_id = data.get("user_id")
    amount = int(data.get("amount", 0))

    # Deduct KPay
    kpay_balances[user_id] -= amount

    # Call Telecom service
    telecom_payload = {"user_id": user_id, "amount": amount}
    tel_resp = requests.post(f"{TELECOM_URL}/add", json=telecom_payload)
    tel_balance = tel_resp.json()["telecom_balance"]

    return jsonify({
        "status": "success",
        "message": f"Payment of {amount} completed for user {user_id}",
        "kpay_balance": kpay_balances[user_id],
        "telecom_balance": tel_balance
    })
from werkzeug.serving import run_simple
run_simple("172.21.13.167", 7001, app, use_reloader=False)
```

Microservice B (Atom) Asynchronous

```python
from flask import Flask, request, jsonify
import requests
from flask_cors import CORS

app = Flask(__name__)
CORS(app)

telecom_balances = {1: 1000, 2: 3000, 3: 0}
CONSUMER_URL = "http://172.21.13.167:7003"

@app.route('/add', methods=['POST'])
def add_balance():
    data = request.get_json()
    user_id = data.get("user_id")
    amount = int(data.get("amount", 0))
    telecom_balances[user_id] += amount
    return jsonify({
        "status": "success",
        "telecom_balance": telecom_balances[user_id]
    })

from werkzeug.serving import run_simple
run_simple("172.21.13.167", 7002, app, use_reloader=False)
```

Microservice C (User) Asynchronous

```
from flask import Flask, request, render_template_string
import requests
from flask_cors import CORS

app = Flask(__name__)
CORS(app)

KPAY_URL = "http://172.21.13.167:7001"

# Store balances in memory for demo
last_result = {}

HTML_PAGE = """
<!DOCTYPE html>
<html>
<head>
    <title>Pay Telecom Bill</title>
</head>
<body>
    <h1>Pay Atom Bill via KPay</h1>
    <form method="POST" action="/pay-bill">
        <label>User ID:</label>
        <input type="number" name="user_id" required><br><br>

        <label>Amount:</label>
        <input type="number" name="amount" required><br><br>

        <button type="submit">Pay Now</button>
    </form>

    {% if result %}
        <h2>Payment Result</h2>
        {% if result.get("message") %}<p>{{ result['message'] }}</p>{% endif %}
        {% if result.get("kpay_balance") %}<p><b>KPay Balance:</b> {{ result['kpay_balance'] }}</p>{%
        {% if result.get("telecom_balance") %}<p><b>Telecom Balance:</b> {{ result['telecom_balance']
    {% endif %}
</body>
</html>
"""
```

```python
@app.route('/')
def home():
    return render_template_string(HTML_PAGE, result=last_result)

@app.route('/pay-bill', methods=['POST'])
def pay_bill():
    user_id = int(request.form.get("user_id"))
    amount = int(request.form.get("amount"))

    # Call KPay service (Service A)
    payload = {"user_id": user_id, "amount": amount}
    resp = requests.post(f"{KPAY_URL}/pay", json=payload)
    result = resp.json()  # This already includes telecom balance from Service A
    return render_template_string(HTML_PAGE, result=result)

from werkzeug.serving import run_simple
run_simple("172.21.13.167", 7003, app, use_reloader=False)
```

Microservice C (User) Asynchronous

1

Model Training

2

Create Flask REST API

3

Predict result

Restful API endpoint in Flask

Home | KNN_Model | KNN_API

localhost:8888/tree/ModelAPI/KNN_Color

jupyter

File View Settings Help

Files Running

Select items to perform actions on them.

New Upload

/ ModelAPI / KNN_Color /

Name	Modified	File Size
templates	35 minutes ago	
KNN_API.ipynb	27 minutes ago	1.9 KB
KNN_Model.ipynb	27 minutes ago	1.8 KB
color.csv	last year	110 B
knnModel.pkl	2 months ago	779 B

Serialize and Deserialize Object using Pickle

The pickle module in Python provides a way to **serialize and deserialize** Python object structures.

Serialization (Pickling)

- Conversion to Byte Stream: **pickle.dump( )**
- Handling Complex Objects
- Storage or Transmission

Deserialization (Unpickling)

- Byte Stream to Object: **pickle.load( )**
- Object Reconstruction

Pickle is designed specifically for Python objects, pickle data generally cannot be directly exchanged with other programming languages.

Fetch Machine Learning Model via API

KNN Model Building

```
import numpy as np
import pandas as pd
import pickle

#Load Dataset
data_set=pd.read_csv('color.csv')

#Split Features and Target
X=data_set[['Brightness','Saturation']]
y=data_set[['Class']]

#Split Training and Testing
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test=train_test_split(X,y,random_state=0)

#Train KNN Model
from sklearn.neighbors import KNeighborsClassifier
knn=KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train.values,y_train.values.ravel())

#Predict
y_predict=knn.predict(X_test)

#Accuracy
from sklearn.metrics import accuracy_score
print("Accuracy Score >> ", accuracy_score(y_test , y_predict)*100)

#Save Model using Pickle
with open("knnModel.pkl", "wb") as model_file:
    pickle.dump(knn, model_file)
```

pickle.dump( )

Serialization

Fetch Machine Learning Model via API

KNN Model API

```python
import numpy as np
import pickle
from flask import Flask, render_template, request

#Create instance of the class
app=Flask(__name__)

#function index()
@app.route("/")
def index():
    return render_template("index.html")

#prediction function
def prediction(features_list):
    features=np.array(features_list).reshape(1,2)
    load_model=pickle.load(open('knnModel.pkl',"rb"))
    target=load_model.predict(features)
    return target[0]

@app.route('/result', methods=['POST'])
def result():
    if request.method=='POST':
        features_list=request.form.to_dict()
        features_list=list(features_list.values())
        features_list=list(map(int, features_list))
        target=prediction(features_list)
    return render_template("result.html", target=target)

from werkzeug.serving import run_simple
run_simple("0.0.0.0", 5000, app, use_reloader=False)
```

Deserialization

pickle.load( )

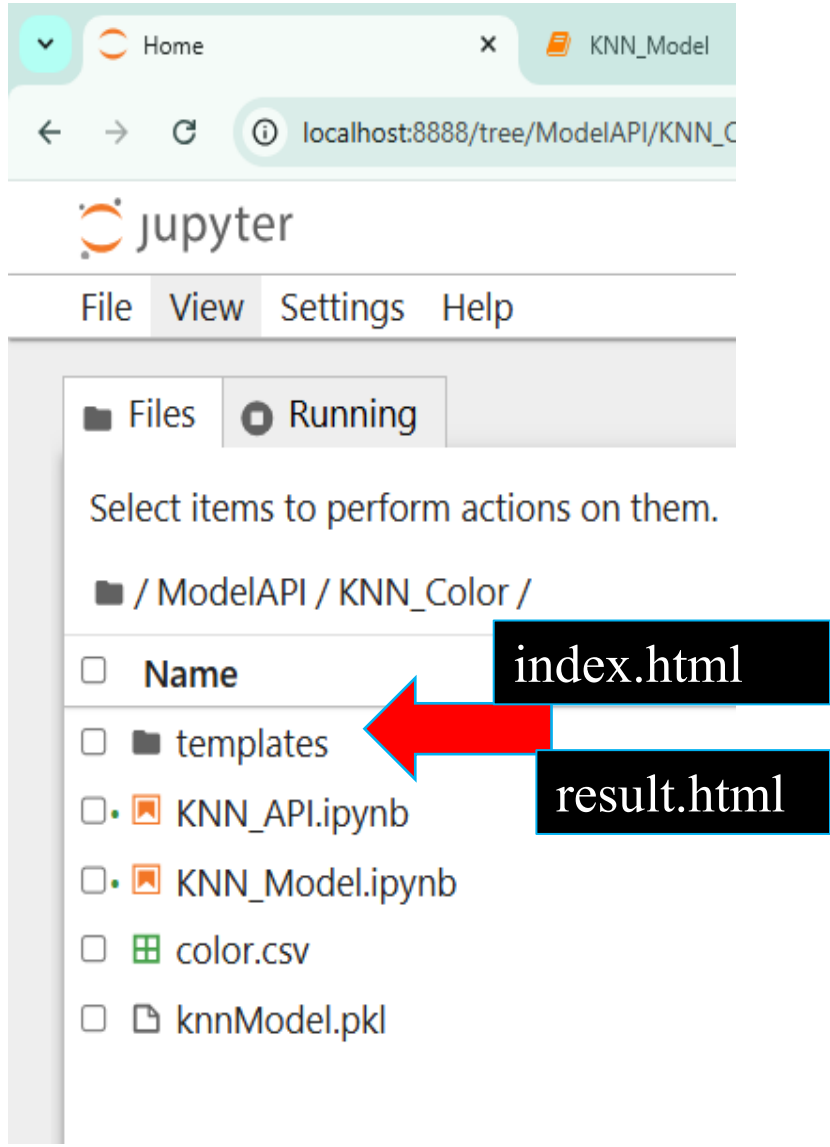
Access Model API via HTML UI

index.html

```
<html>
<body>
    <h3>Color Feature</h3>

<div>
  <form action="/result" method="POST">

      <label for="brightness">Brightness </label>
      <input type="text" id="brightness" name="brightness">
      <br>

      <label for="saturation">Saturation </label>
      <input type="text" id="saturation" name="saturation">
      <br>

    <br>
    <input type="submit" value="Submit">
  </form>
</div>
</body>
</html>
```

result.html

```
<!doctype html>
<html>
   <body>
       <h1> {{ target }}</h1>
   </body>
</html>
```



Accessing or Fetch Real World API

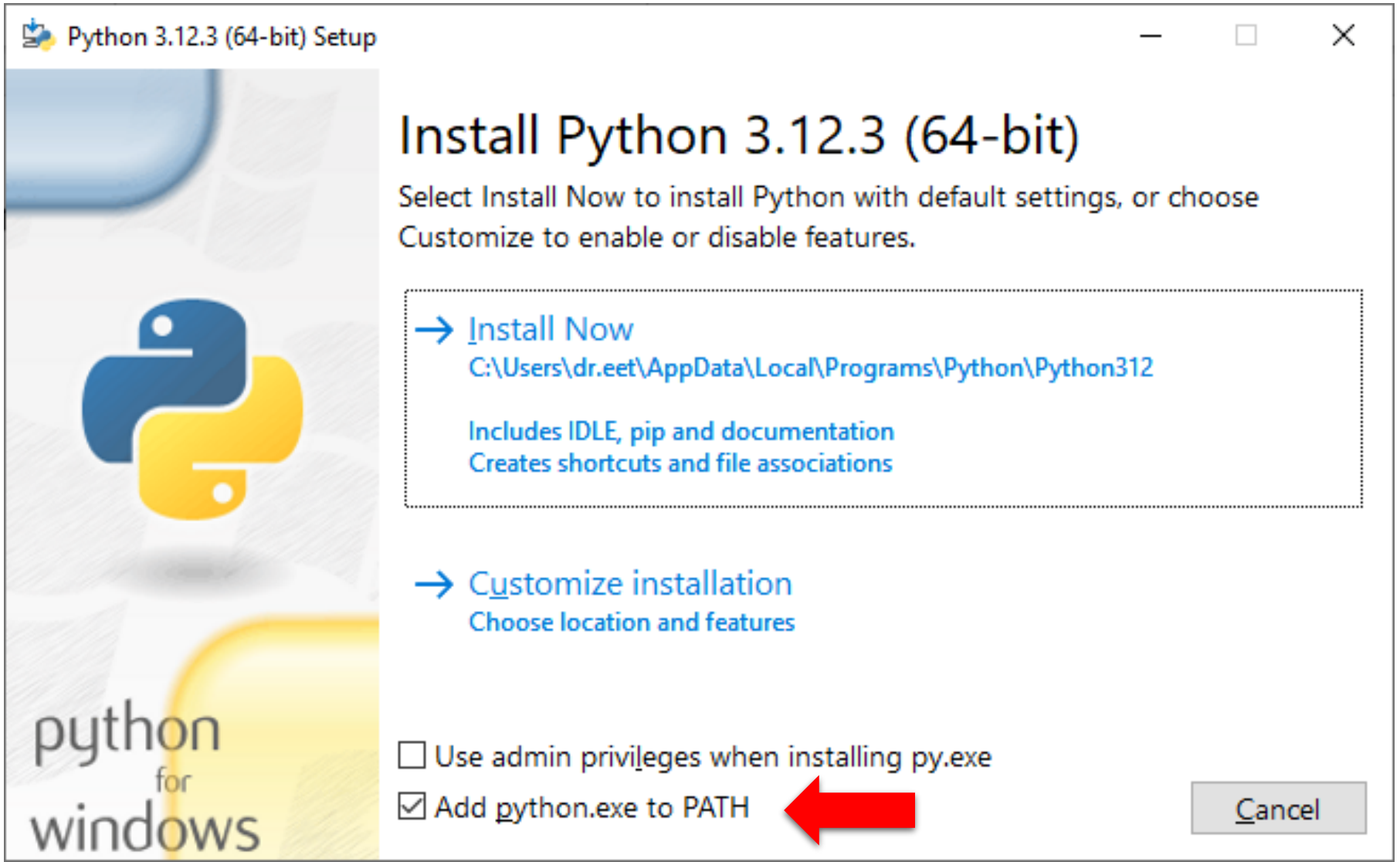
[1]:
```
import requests

resp = requests.get("https://catfact.ninja/fact", headers={"User-Agent": "example"}, timeout=10)
resp.raise_for_status()
data = resp.json()
print(data["fact"])
```

```
Cats are North America's most popular pets: there are 73 million cats compared to 63 million dogs. Over
30% of households in North America own a cat.
```

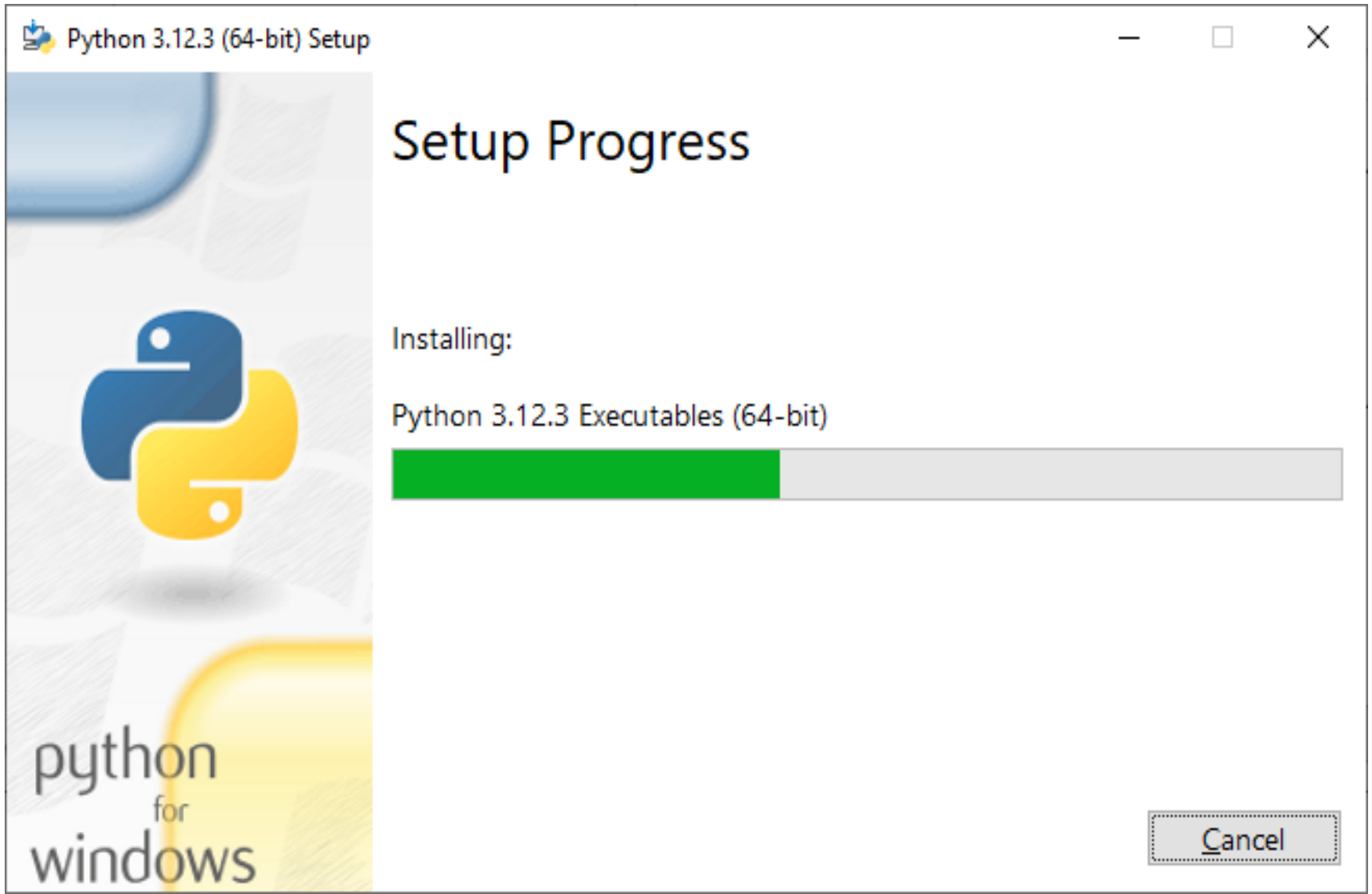
Configuring Jupyter notebook for Python

Step 1: Install Python

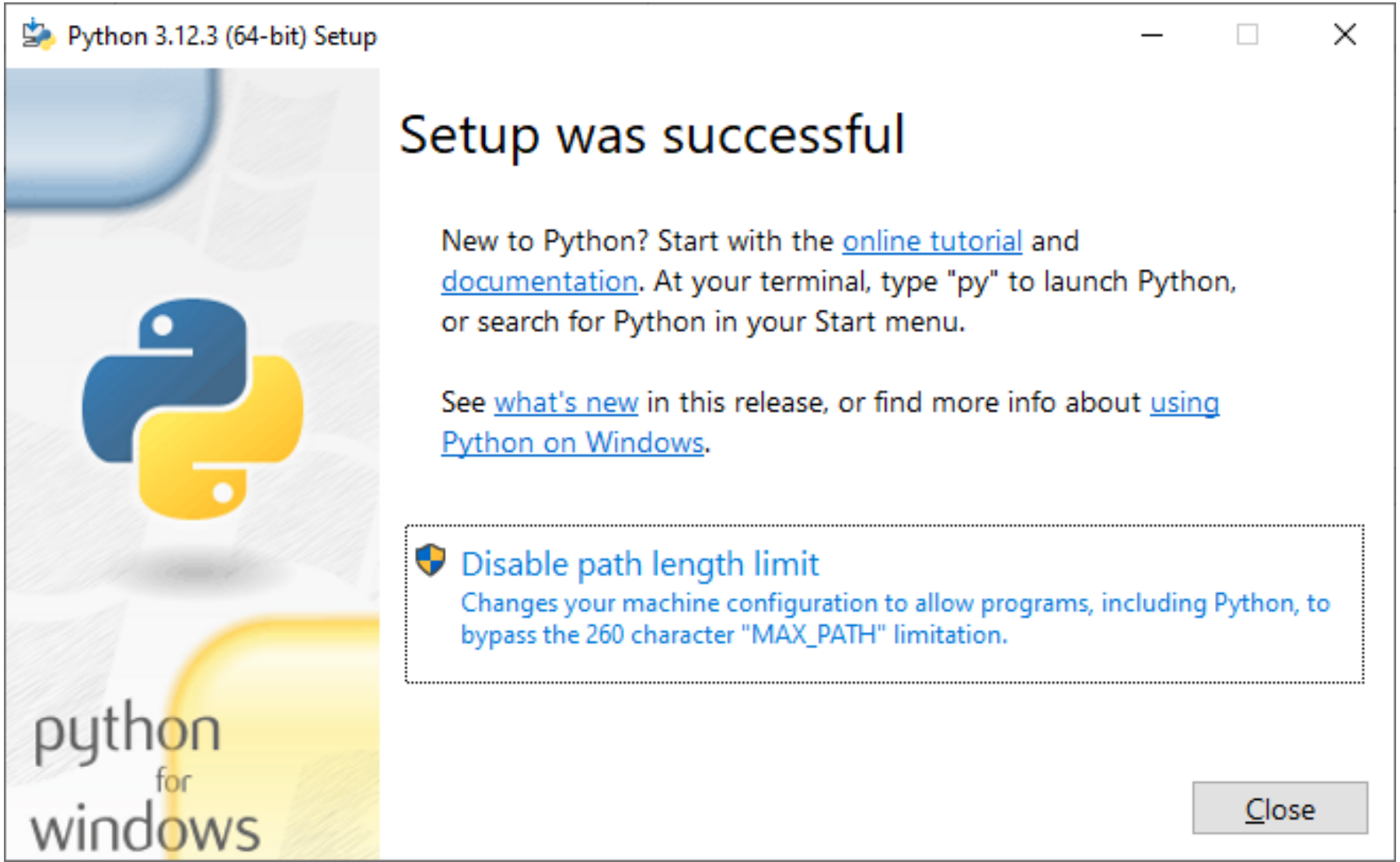
https://www.python.org/downloads/



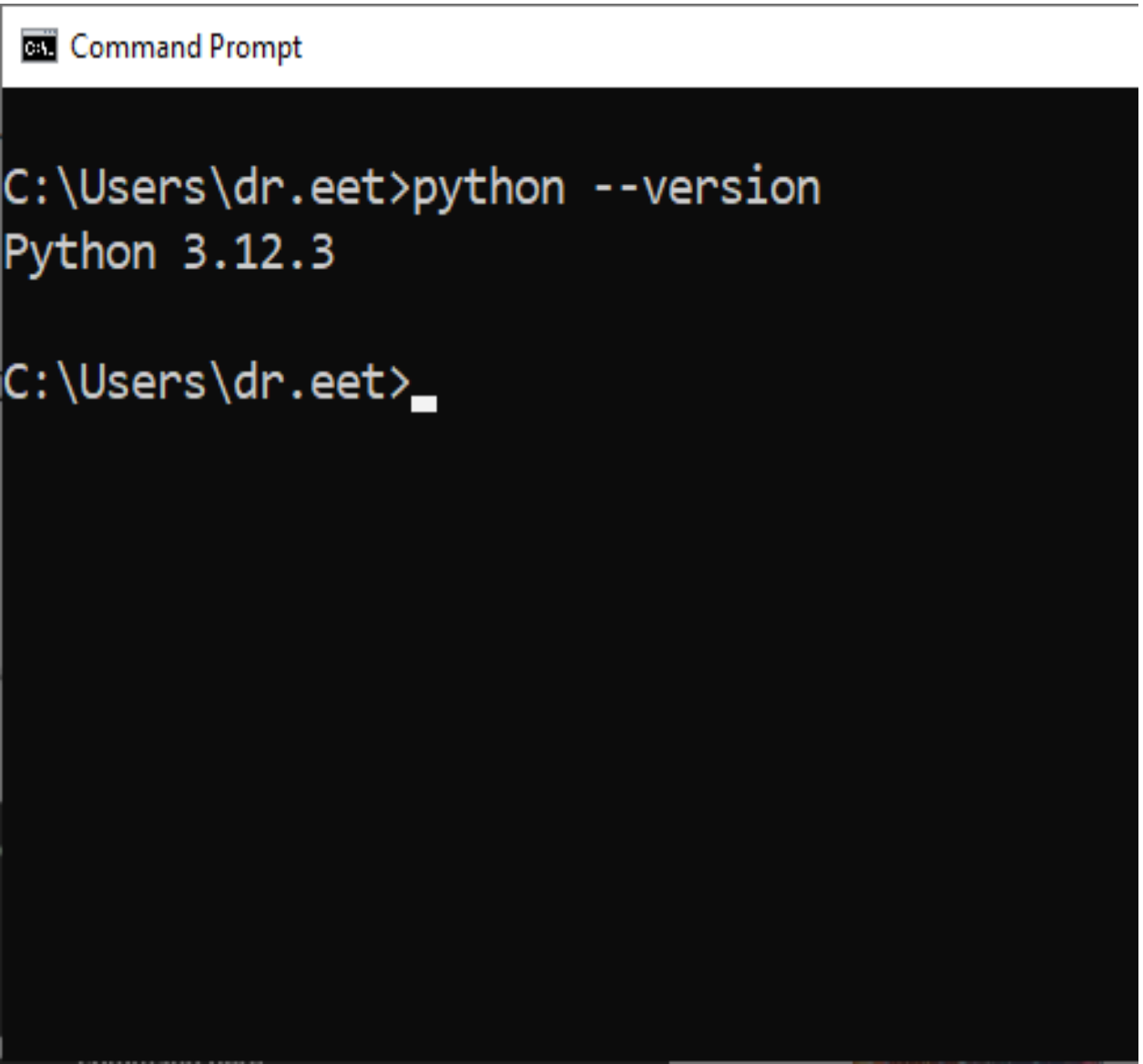
Step 1: Install Python -> Setup Progress



Step 1: Install Python -> Setup was Successful



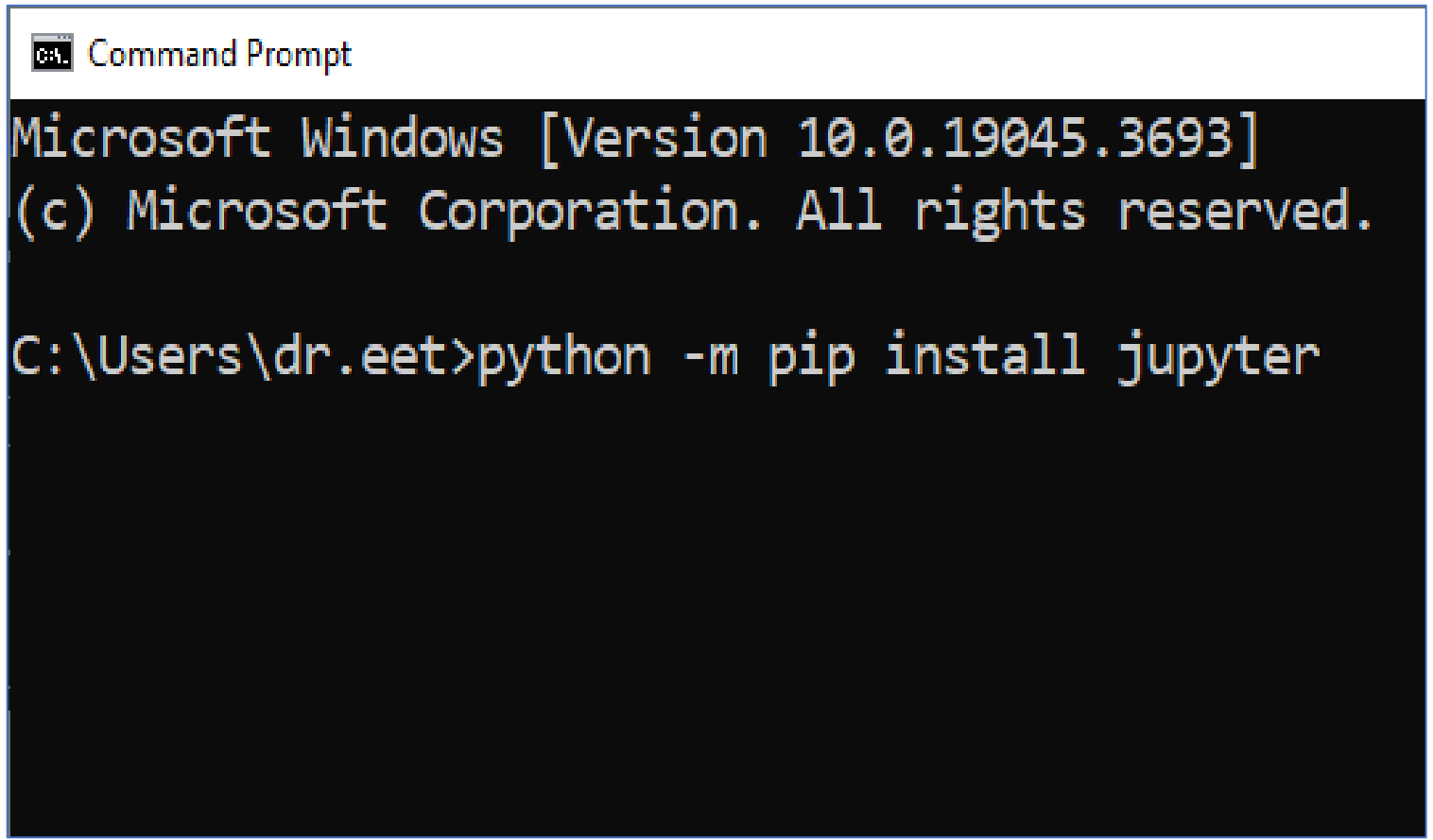
Step 2: Install Jupyter Notebook via PIP



1. Open Command Prompt by searching ‘cmd’ in the Windows search bar.
2. Check if Python is installed by running the ‘python - -version’ command.

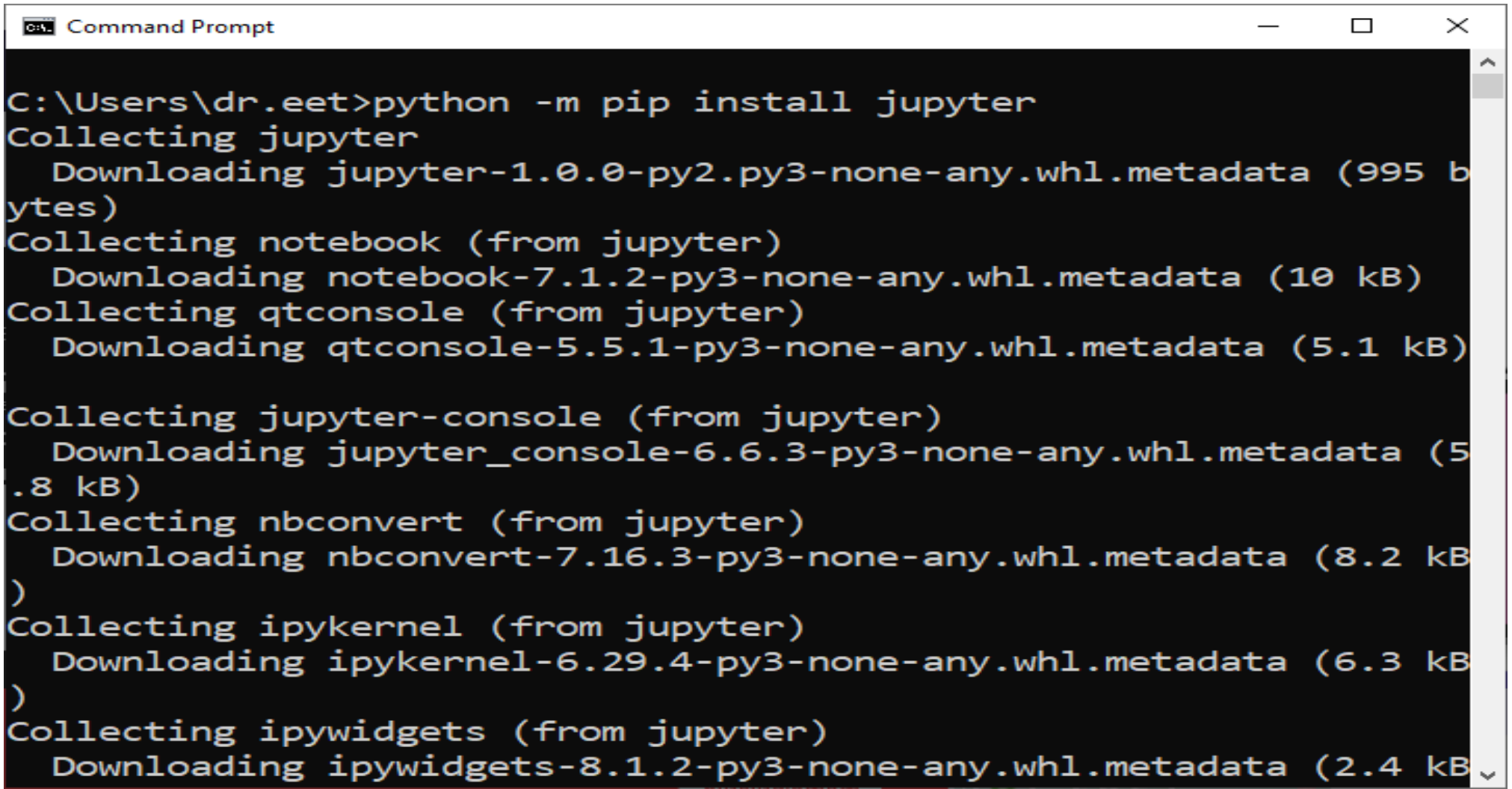
Step 2: Install Jupyter Notebook via PIP

1. To install Jupyter Notebook via pip, run the command ‘ python –m pip install jupyter ’

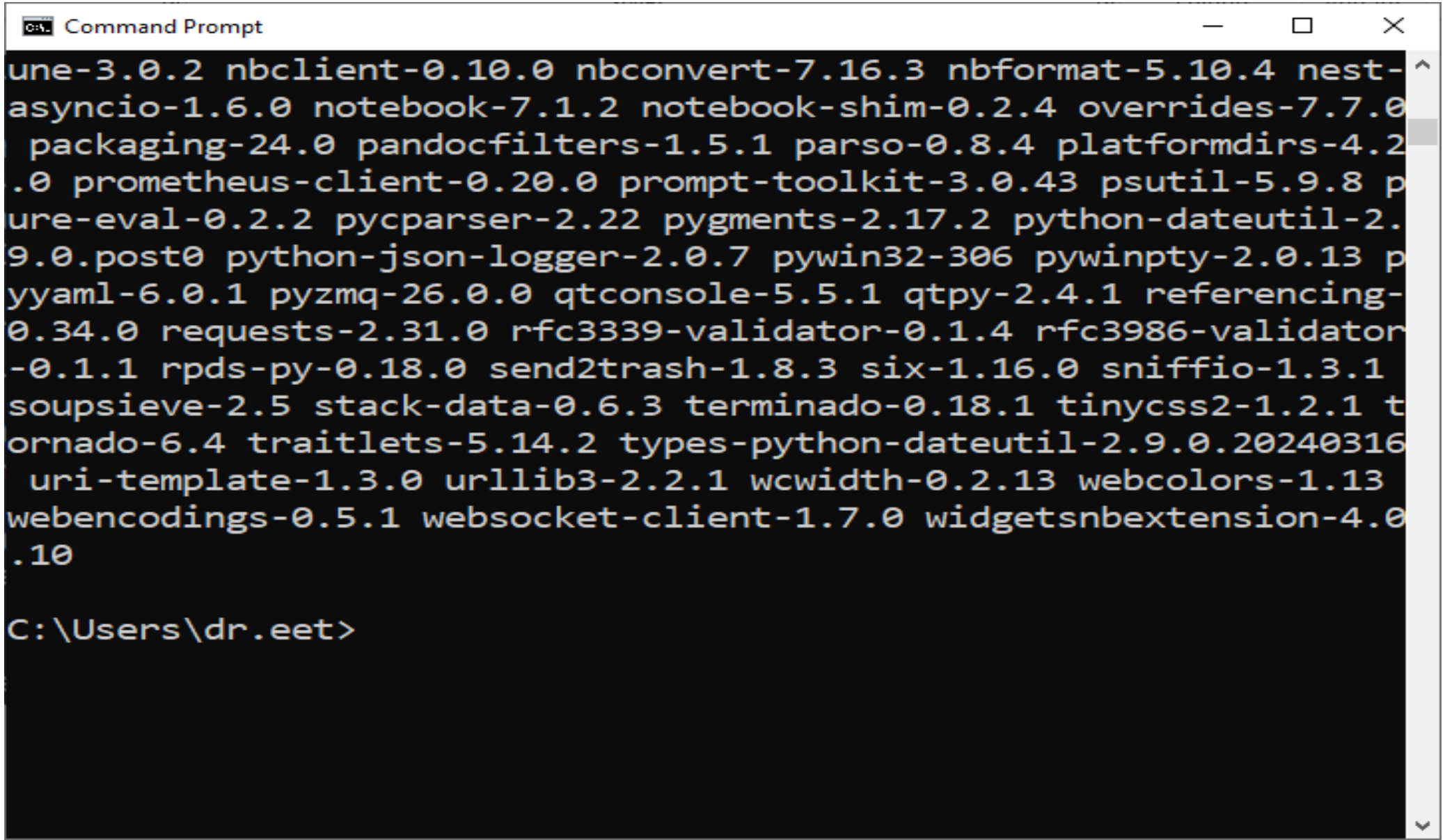


Step 2: Install Jupyter Notebook -> Installation Progress

1. Please wait to complete the installation process

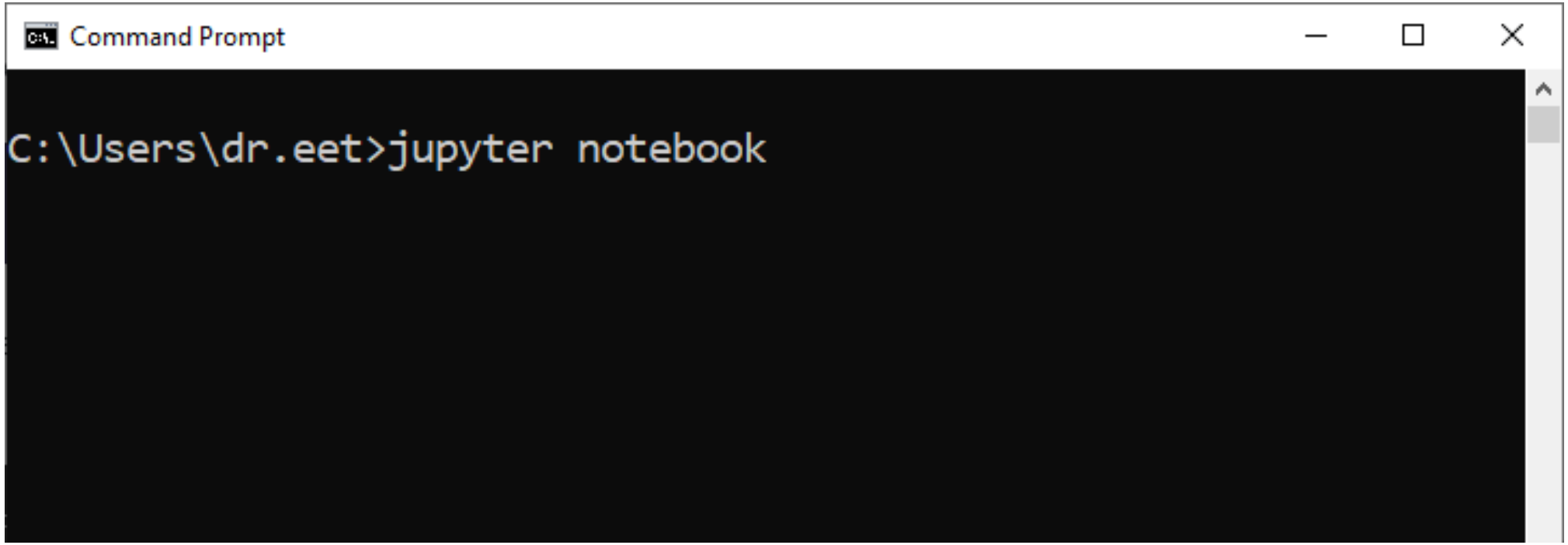


Step 2: Install Jupyter Notebook -> Installation Success



Step 3: Access Jupyter Notebook

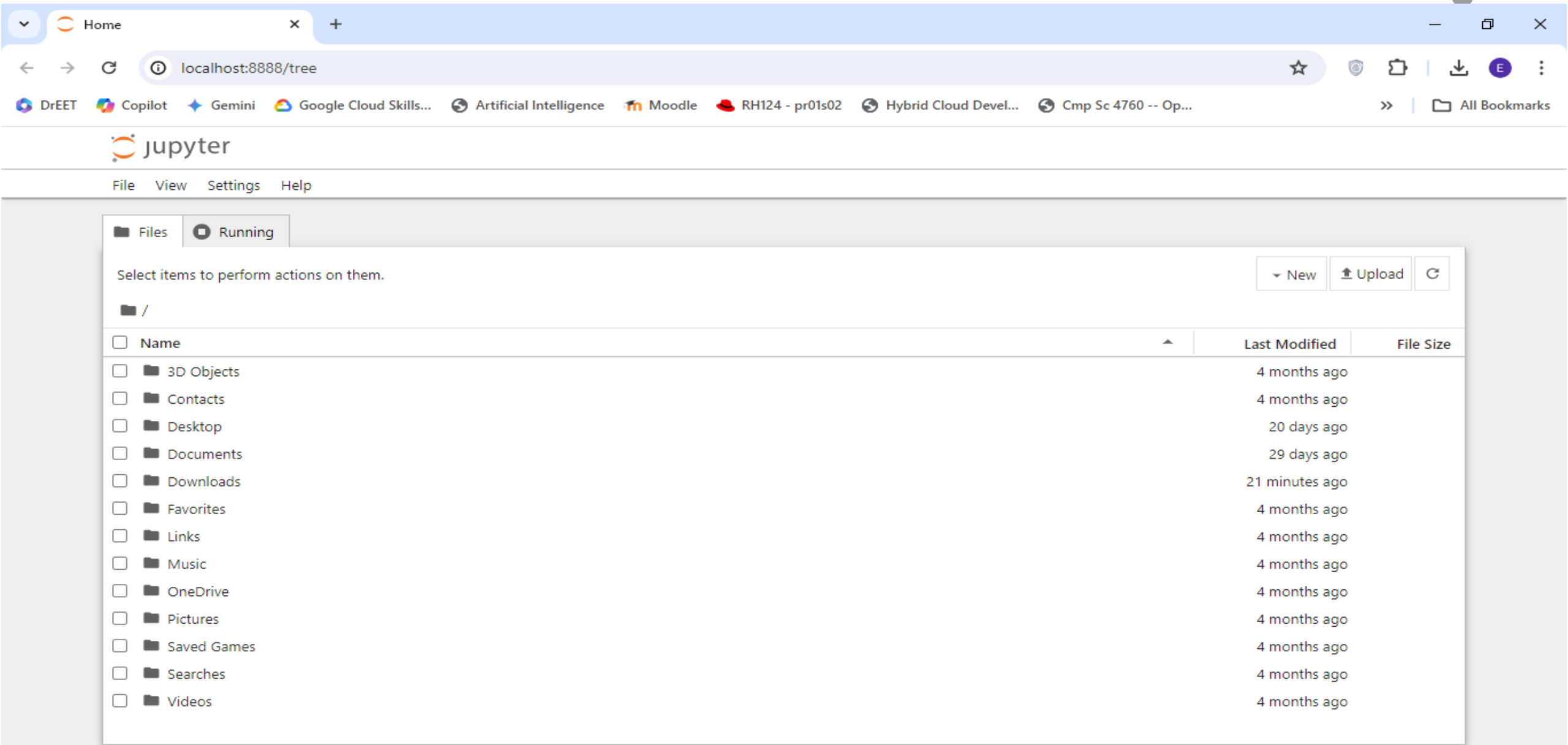
To access Jupyter Notebook via command prompt, run the command ‘ jupyter notebook ’



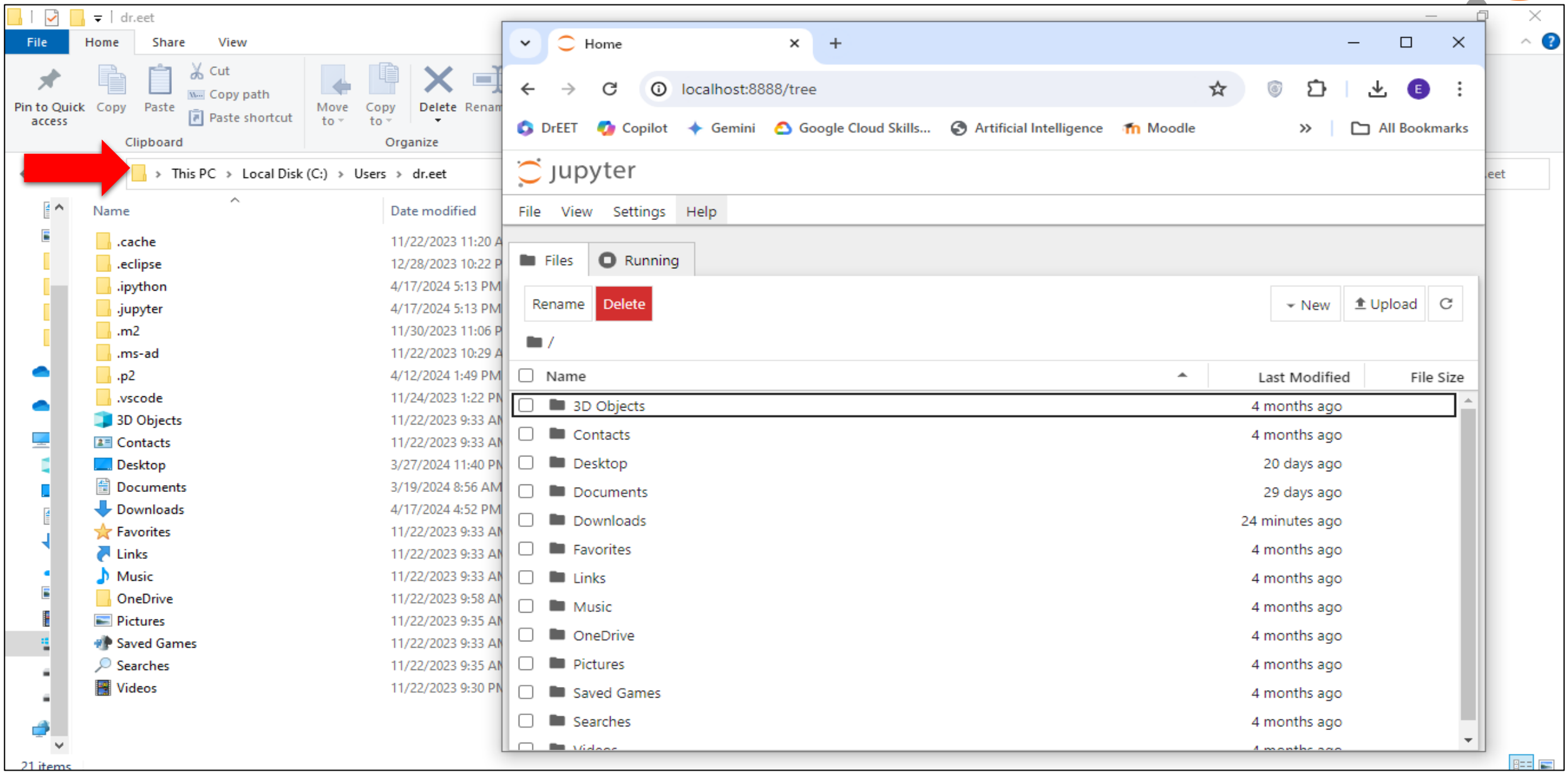
နောက်တခါ Jupyter Notebook ပြန်ဖွင့်ချင်တိုင်း Command Prompt ကို ဖွင့်ပြီး ဒီနေရာကနေပြန်စရပါမယ်

ရှေ့က Installation Process တွေ ပြန်လုပ်စရာမလိုပါ

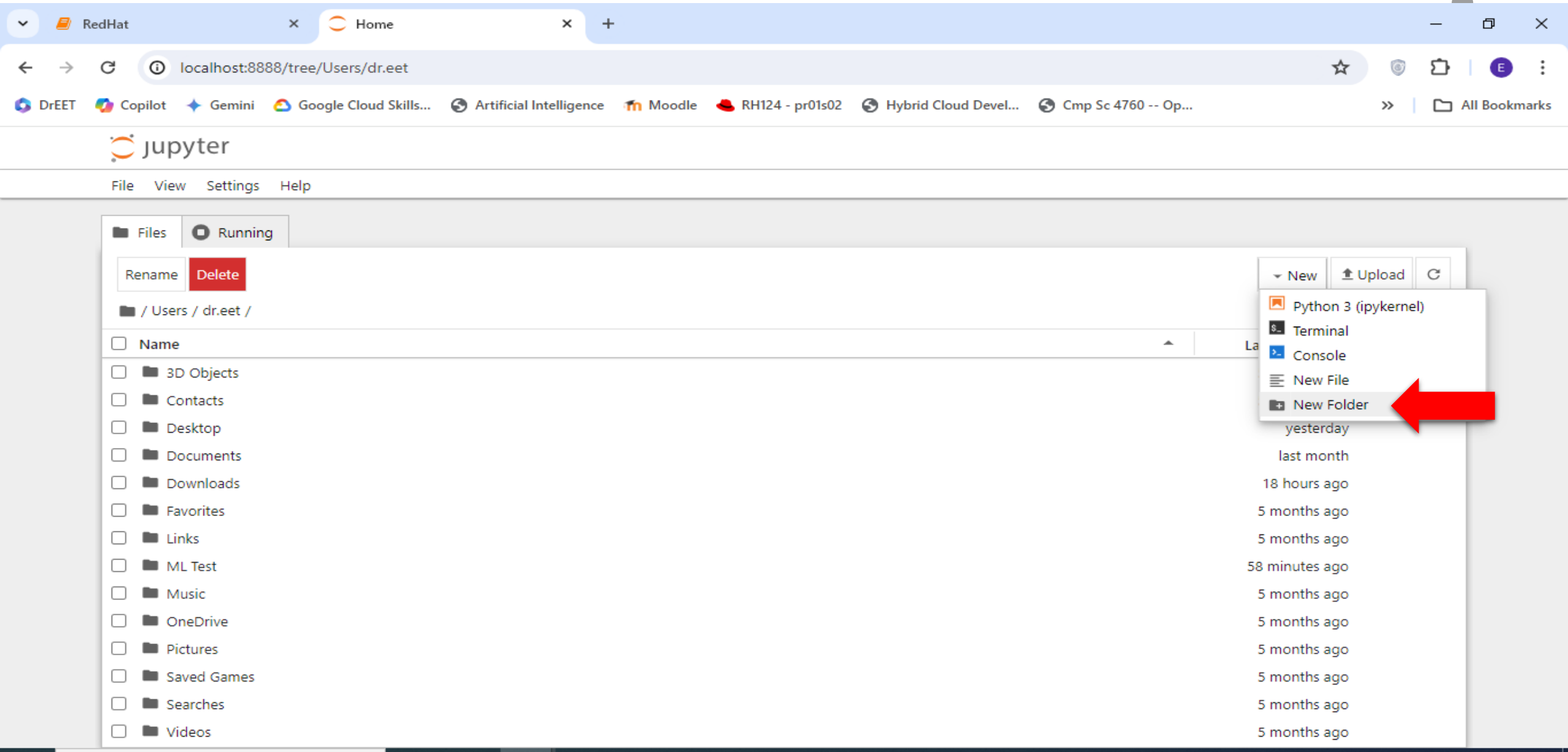
The Jupyter Notebook will open via Browser.

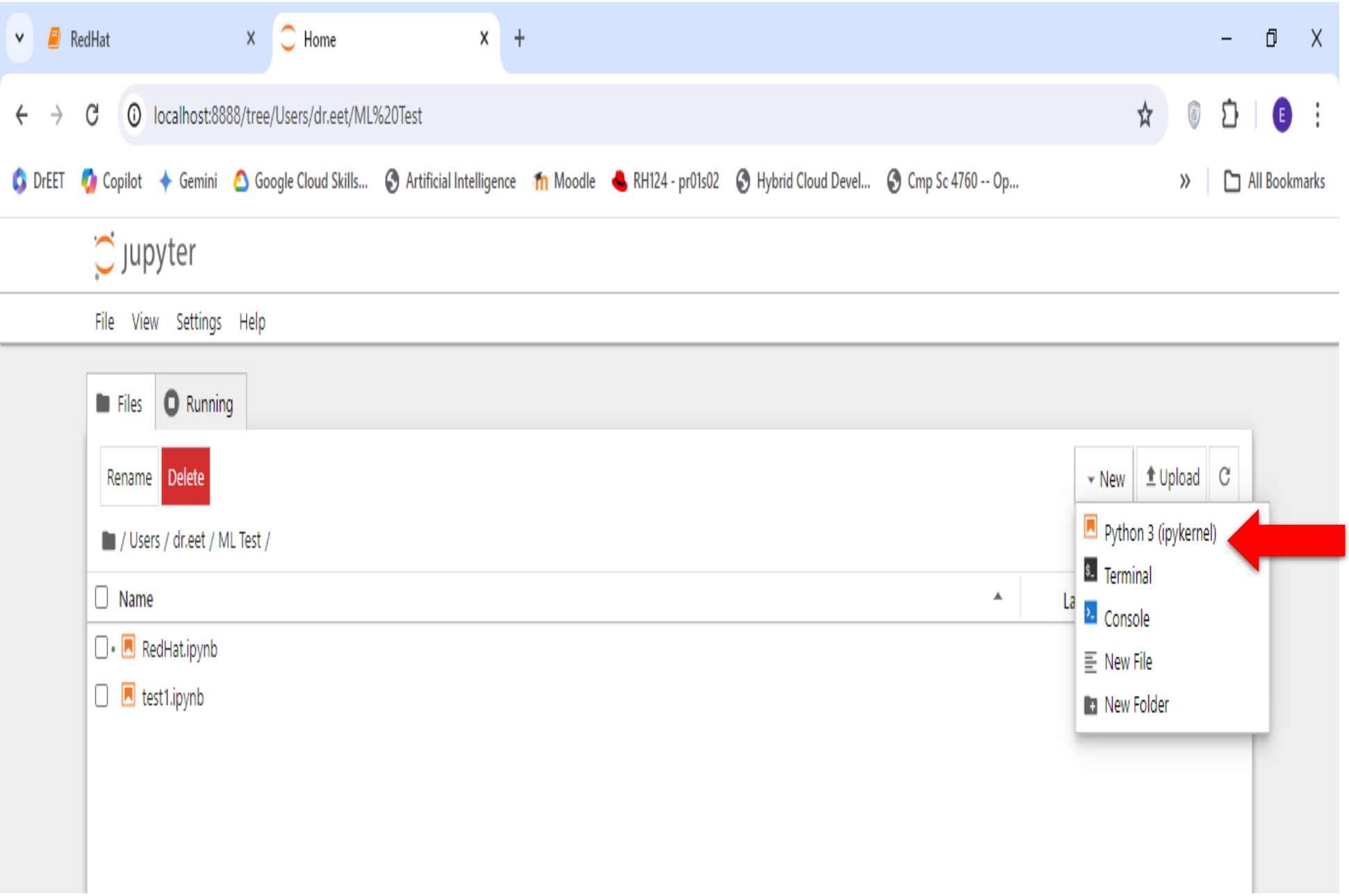


You can access the Python program file via Jupyter Notebook Workspace



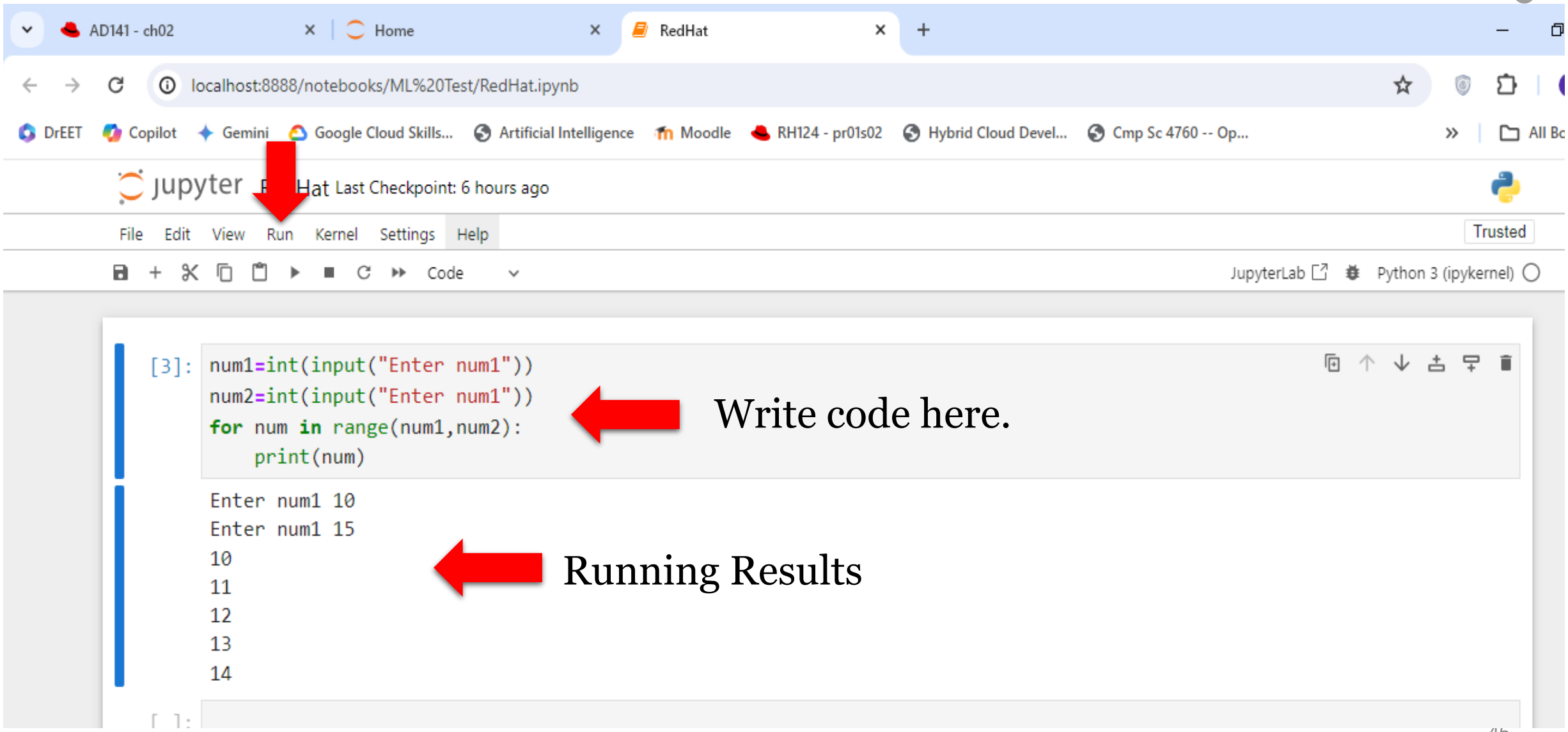
Create a new folder to store Python programs.





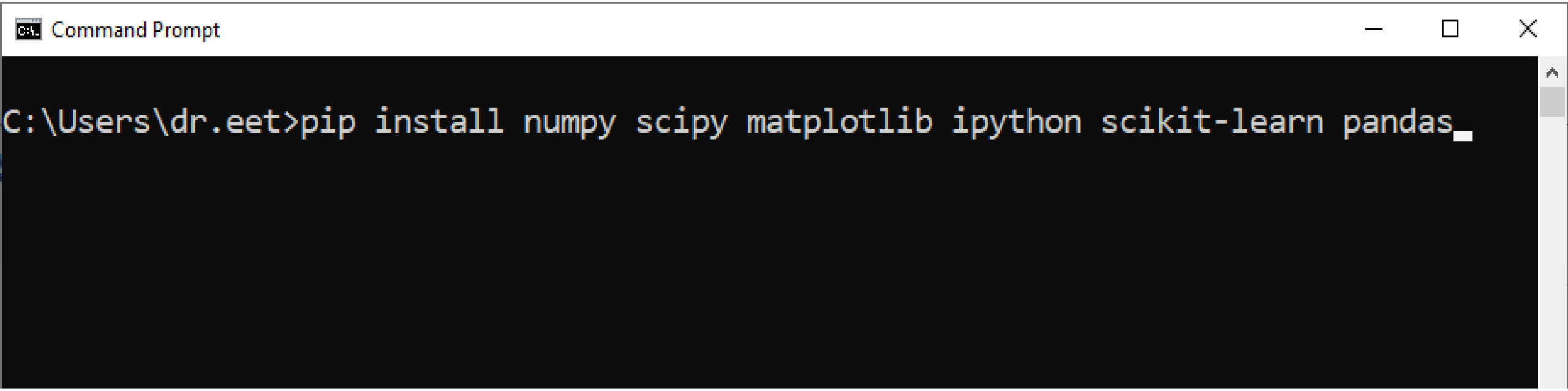
Create a new Python file from Python 3

You can easily write and run Python code in Jupyter Notebook.



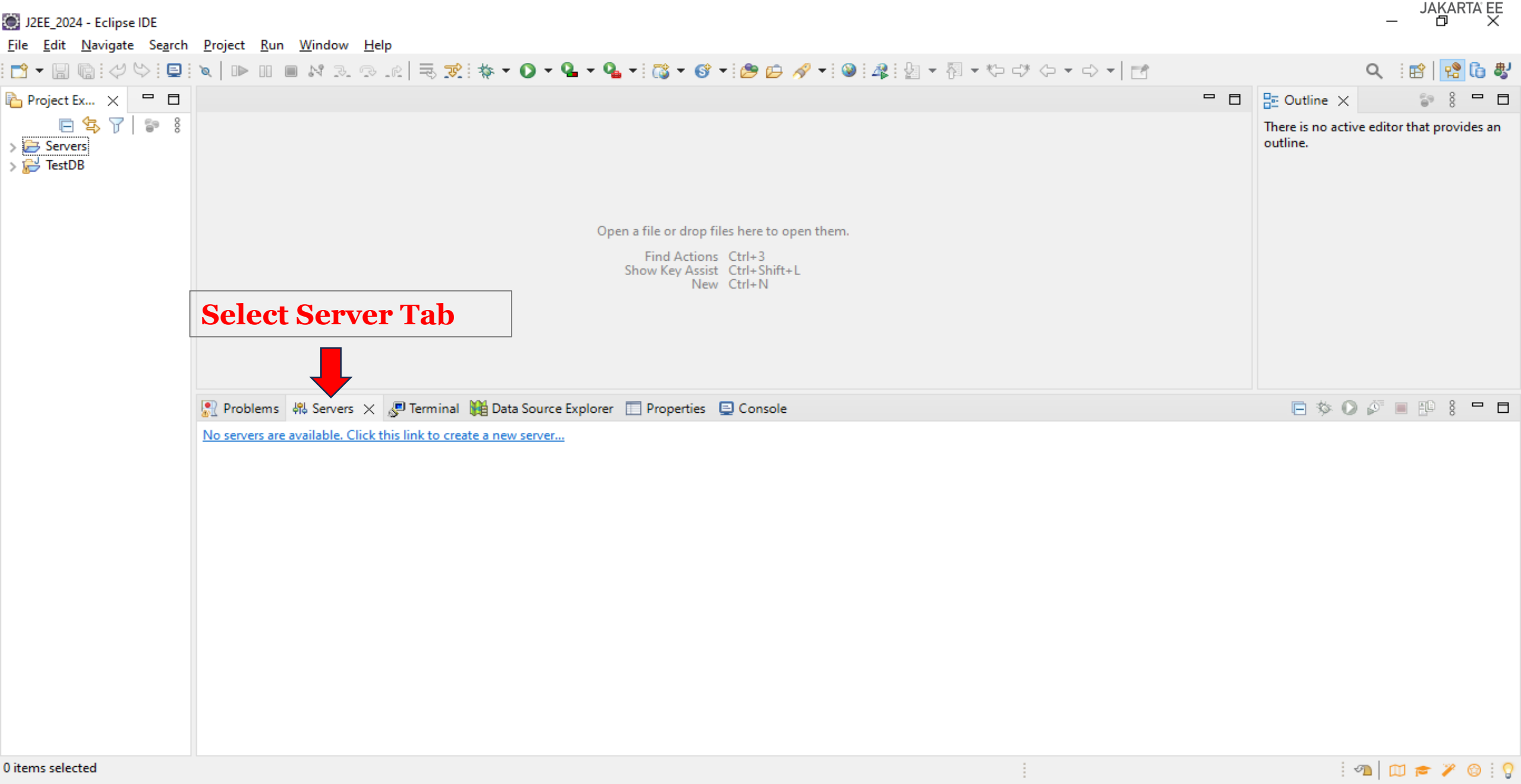
Installing scikit-learn

```
pip install numpy scipy matplotlib ipython scikit-learn pandas
```

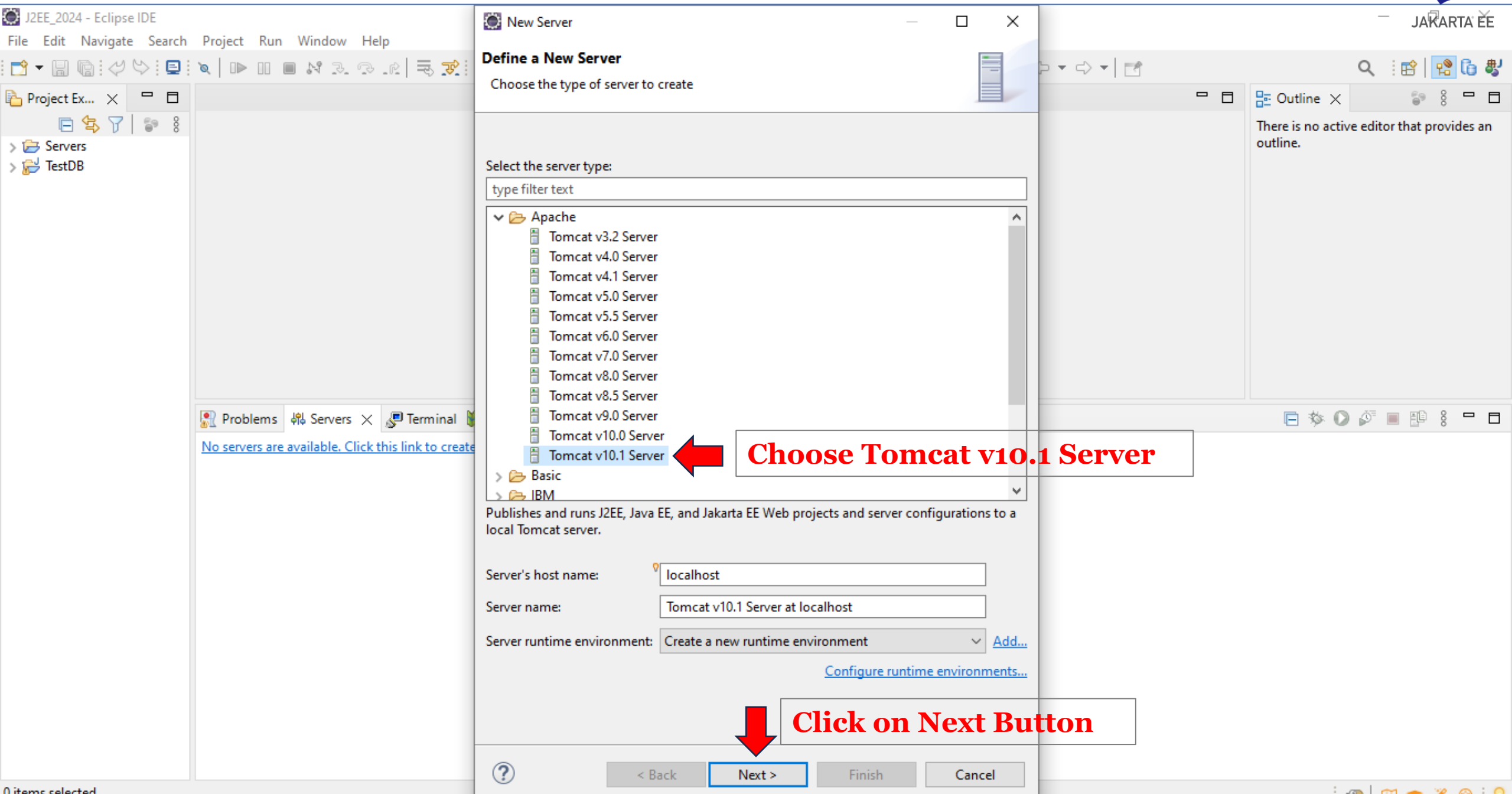


I. Configure Tomcat Web Server

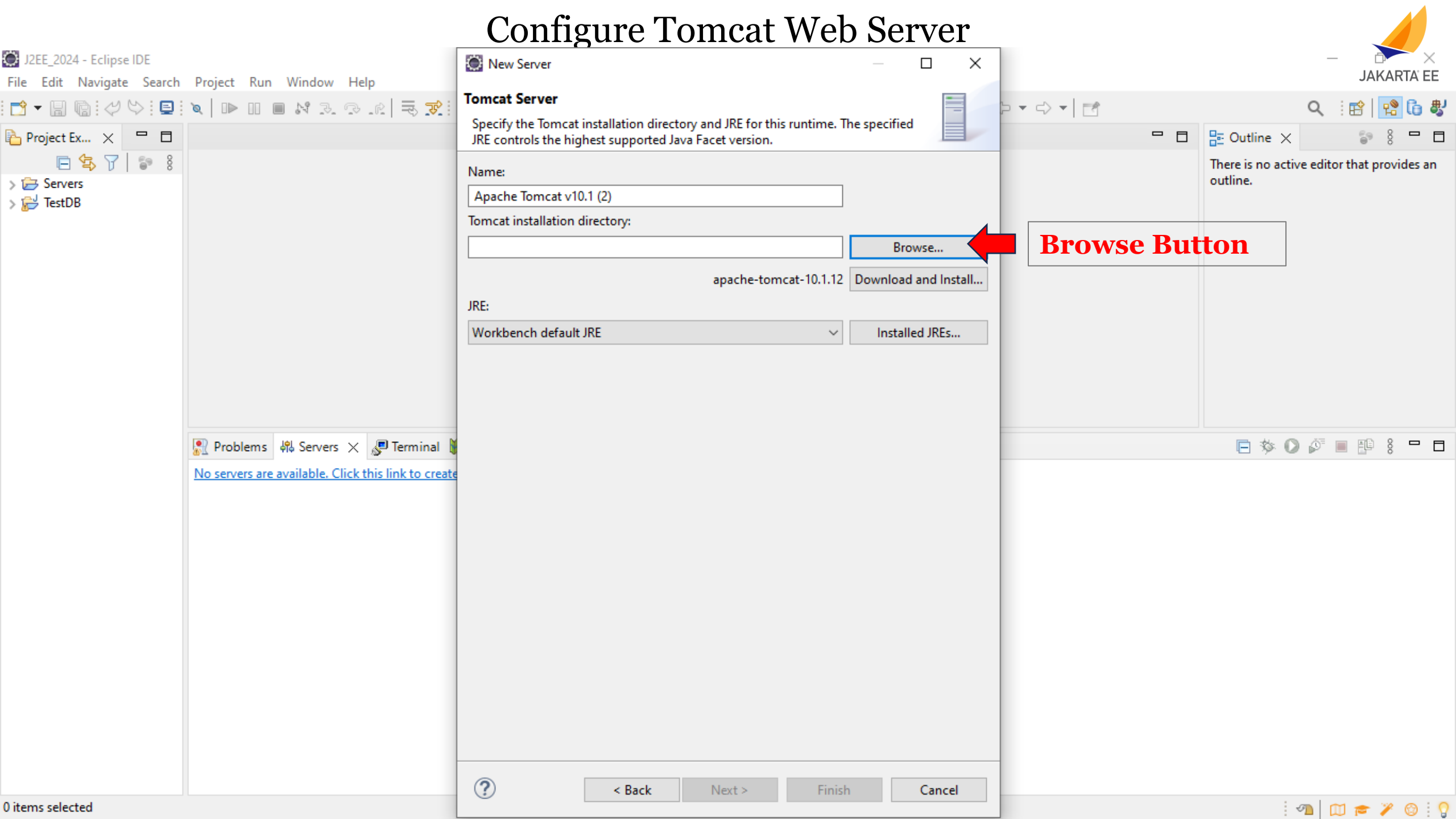
Configure Tomcat Web Server



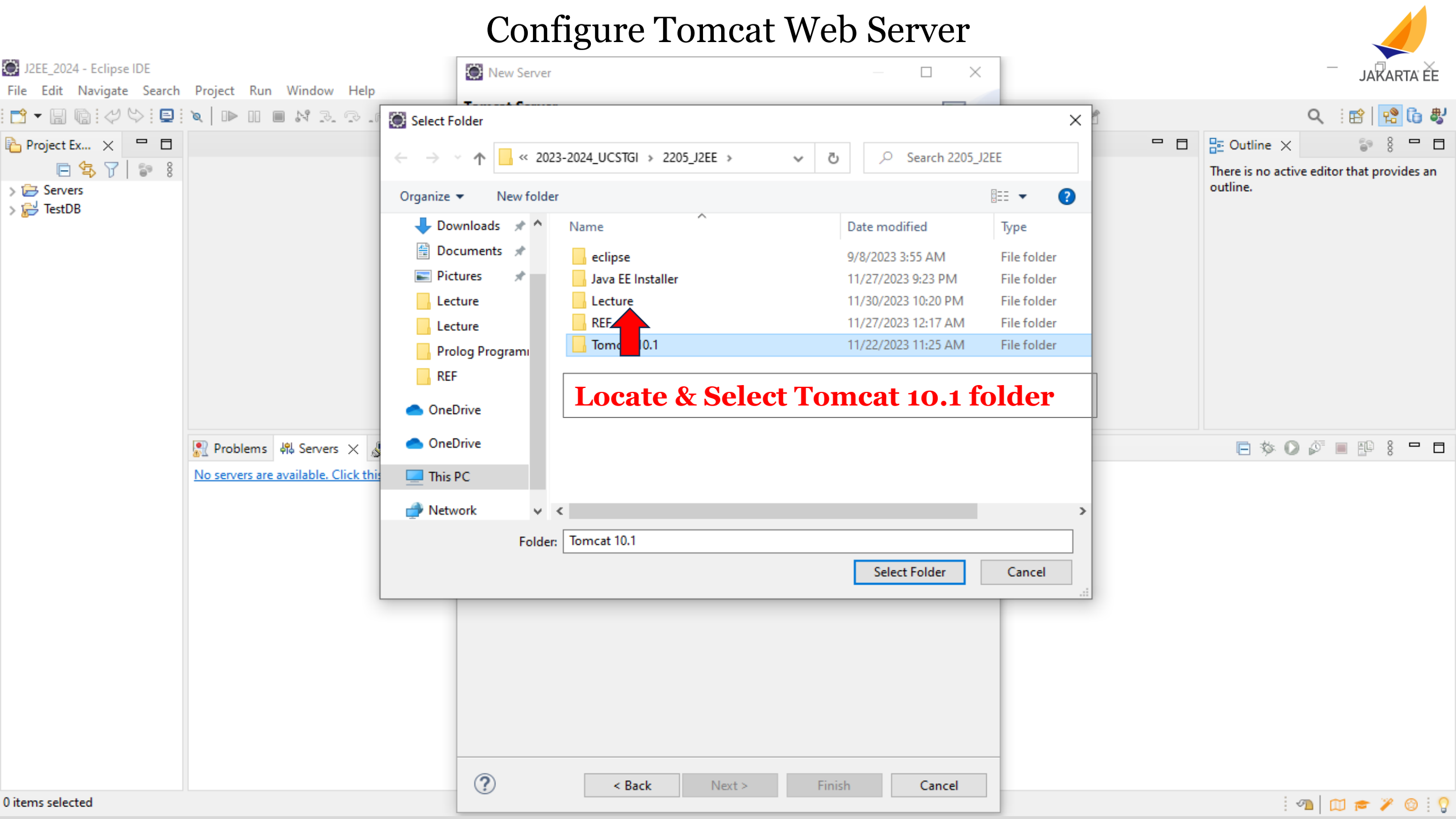
Configure Tomcat Web Server



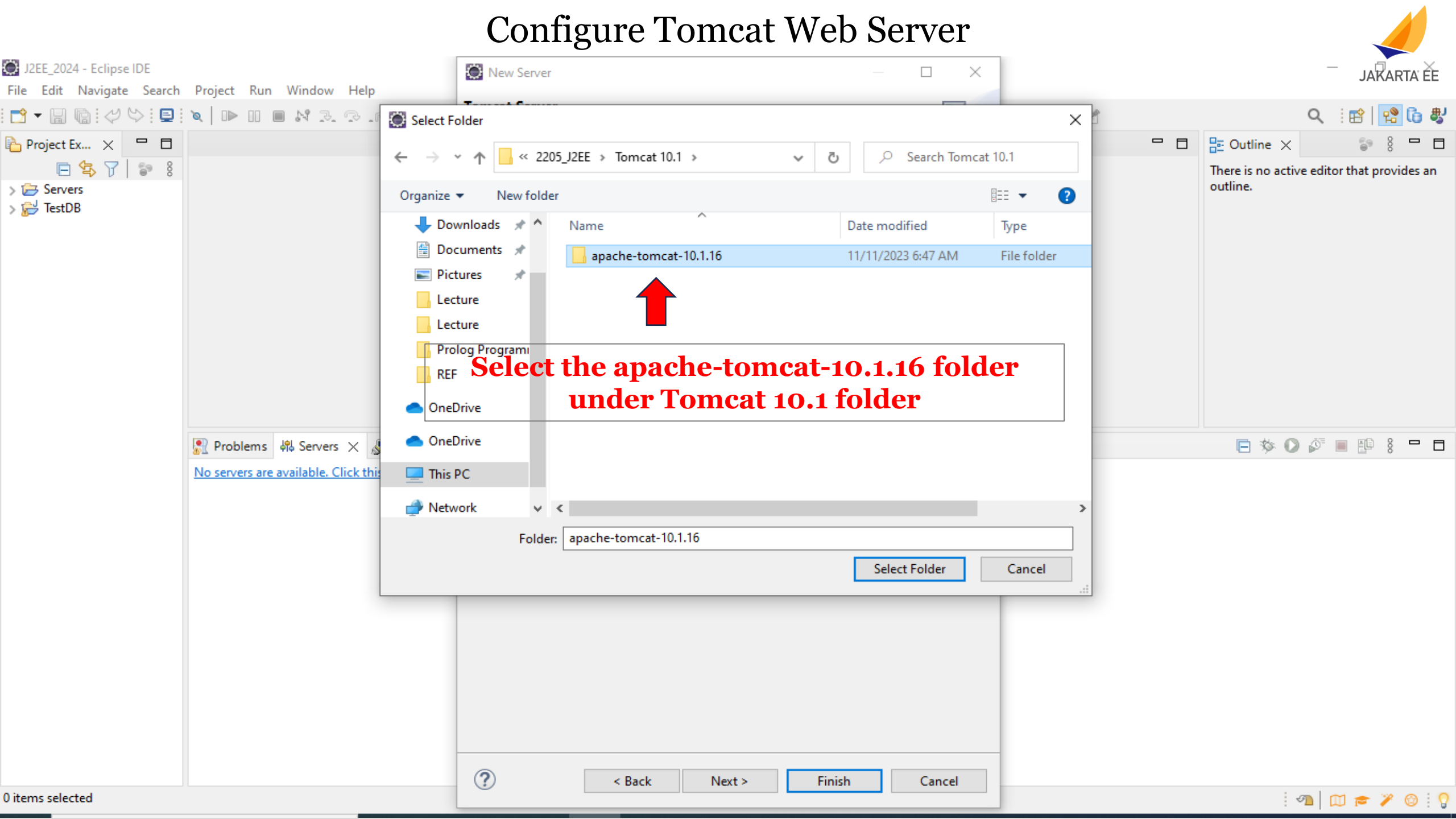
Configure Tomcat Web Server



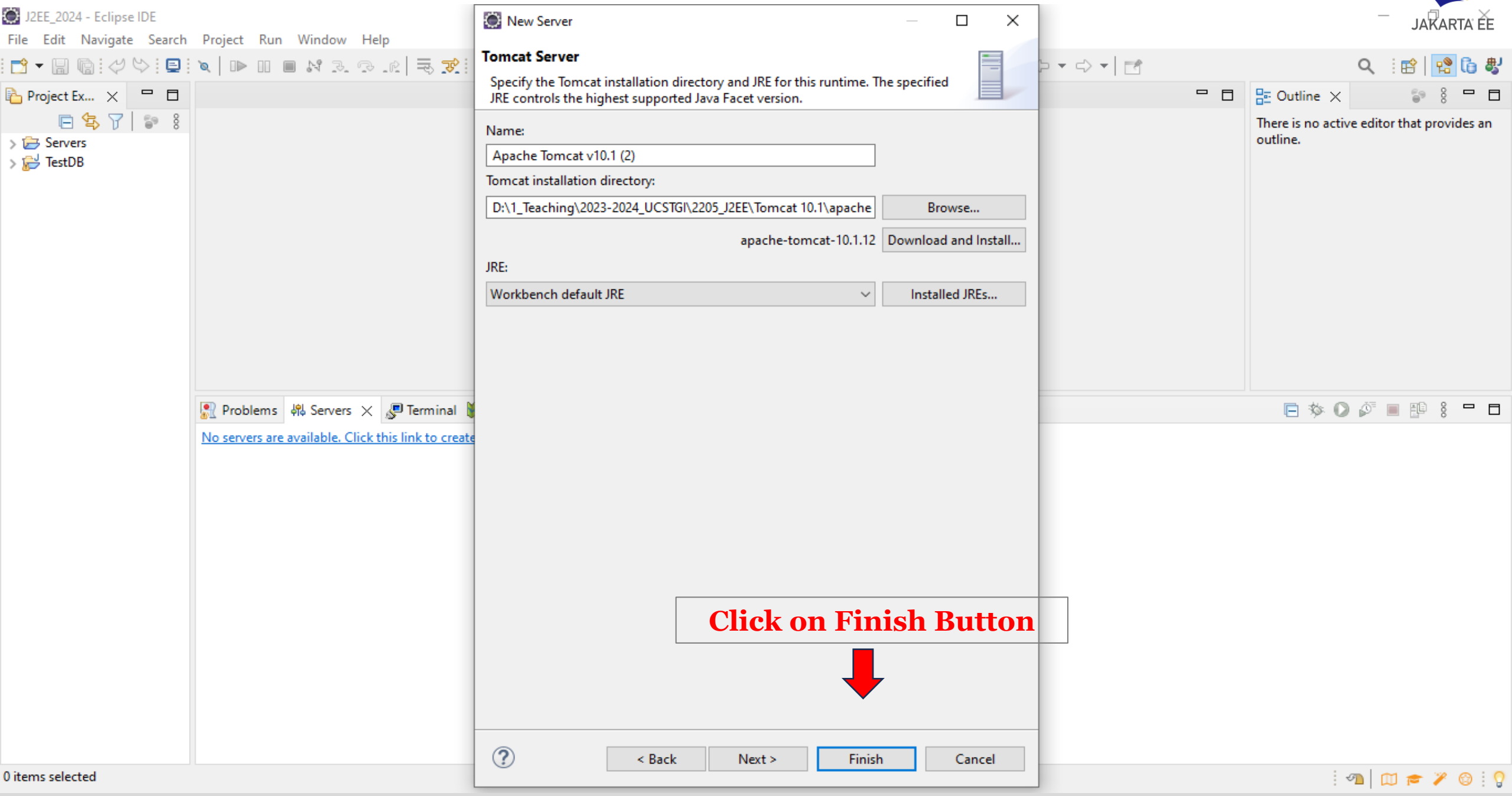
Configure Tomcat Web Server



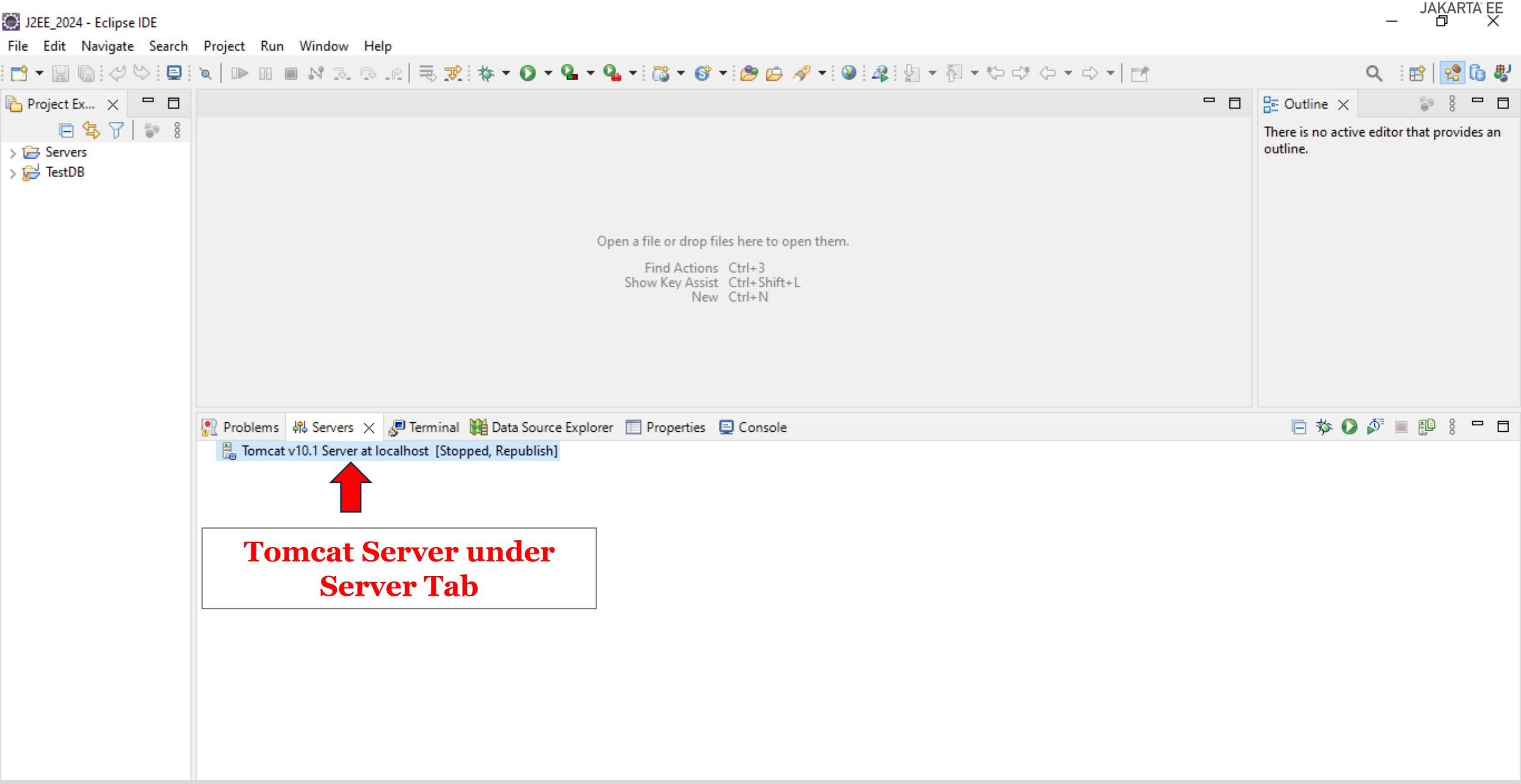
Configure Tomcat Web Server



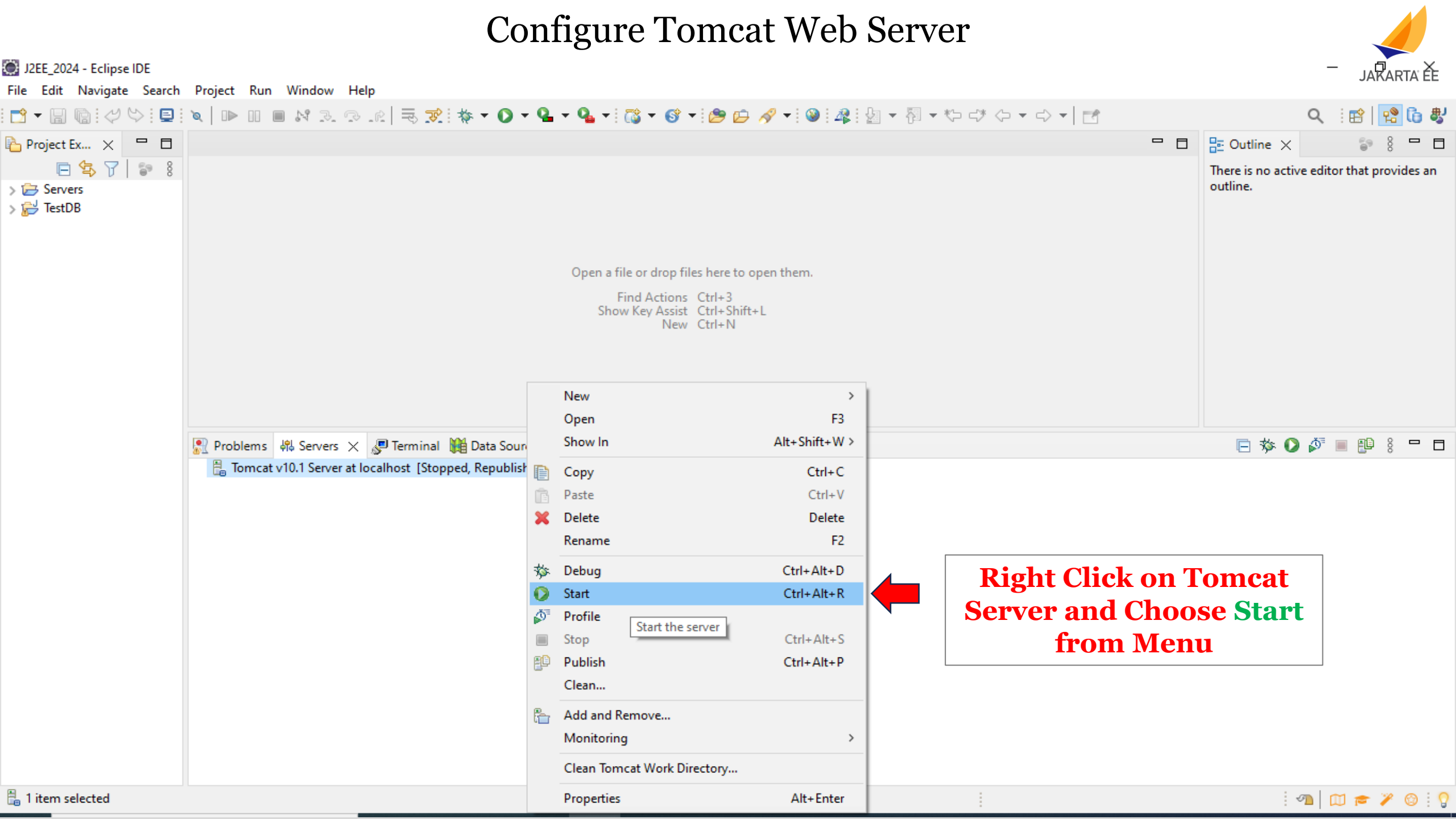
Configure Tomcat Web Server



Configure Tomcat Web Server

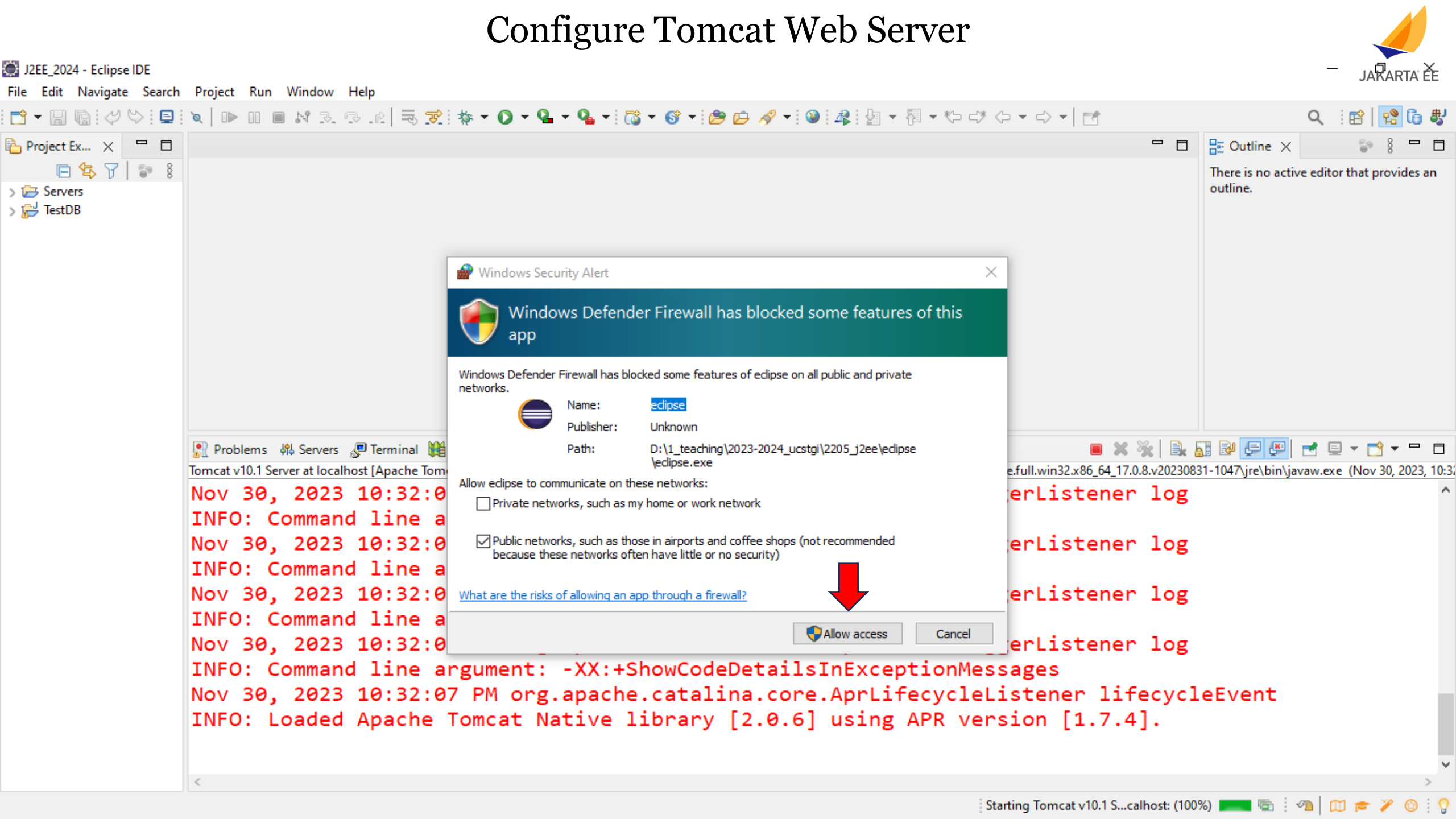


Configure Tomcat Web Server

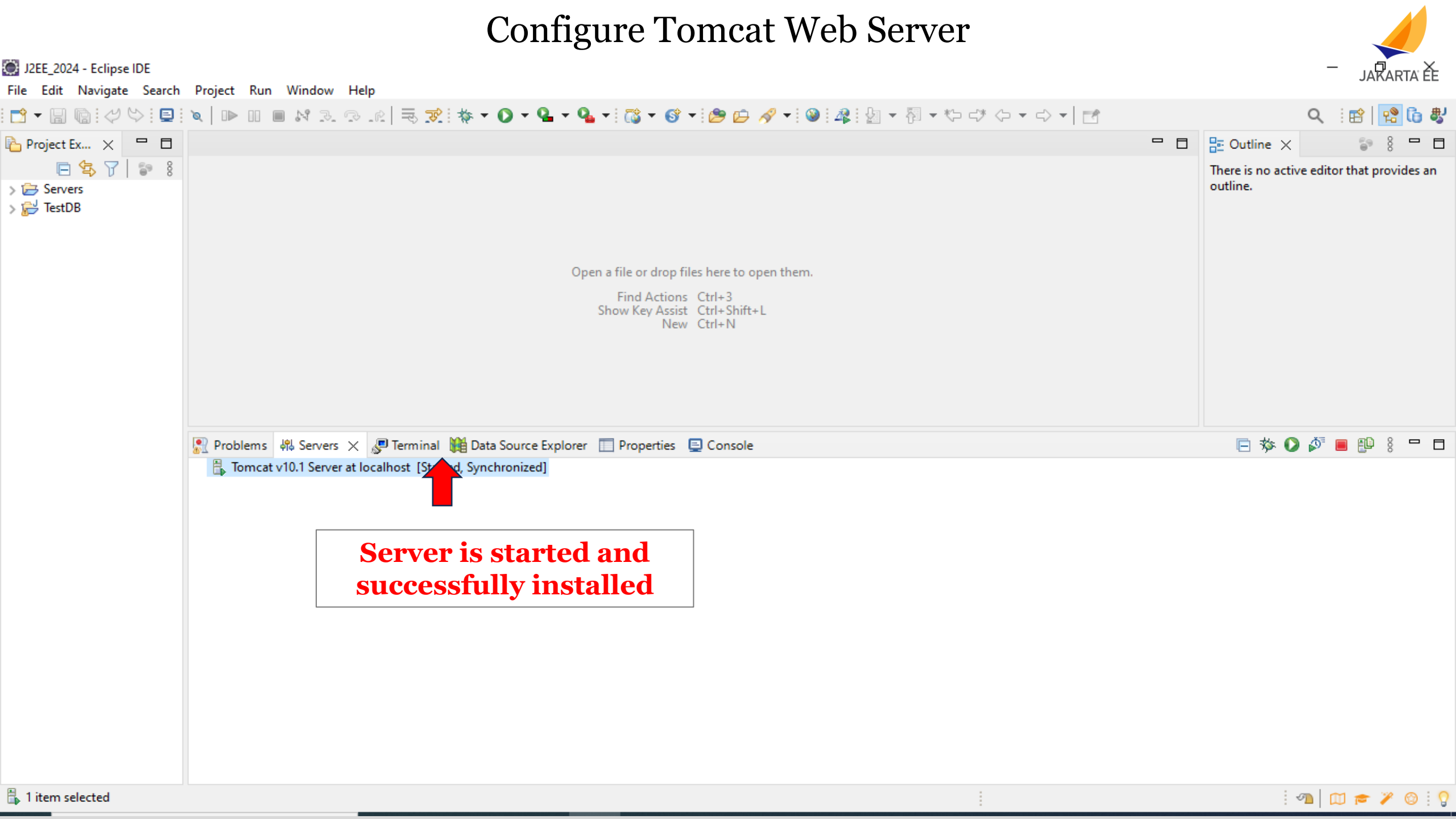


Right Click on Tomcat Server and Choose Start from Menu

Configure Tomcat Web Server

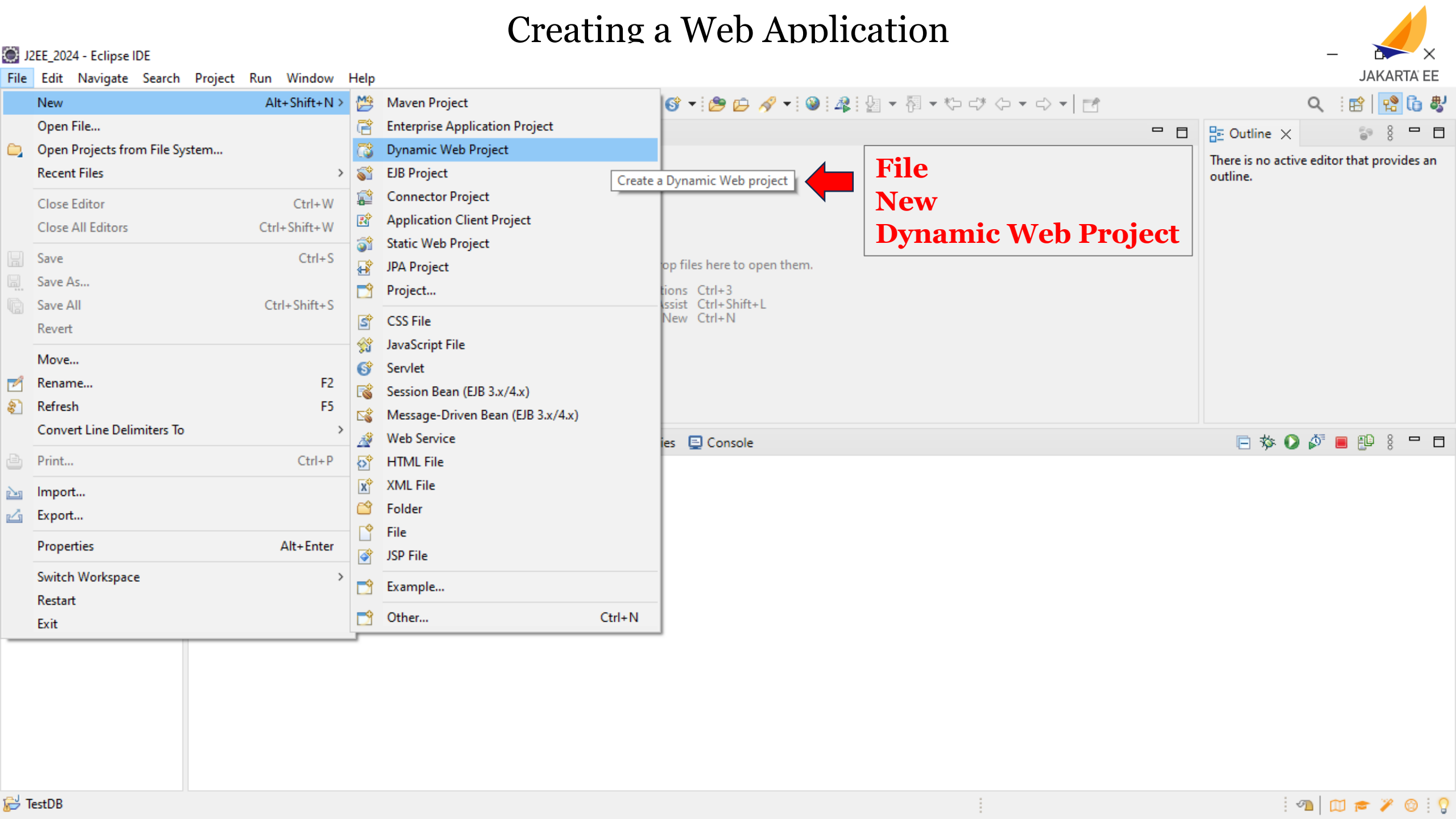


Configure Tomcat Web Server

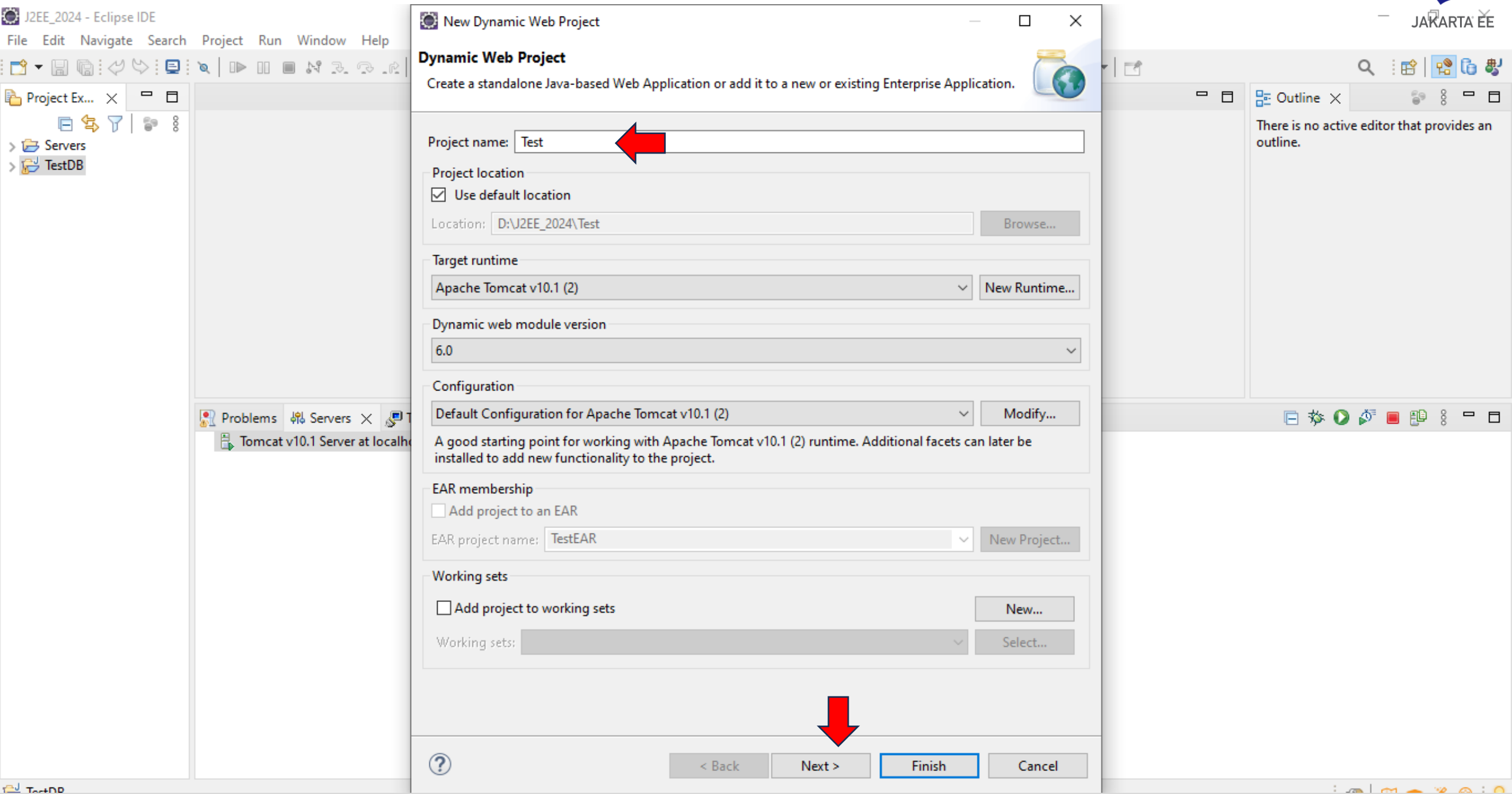


II. Creating a Dynamic Web Application

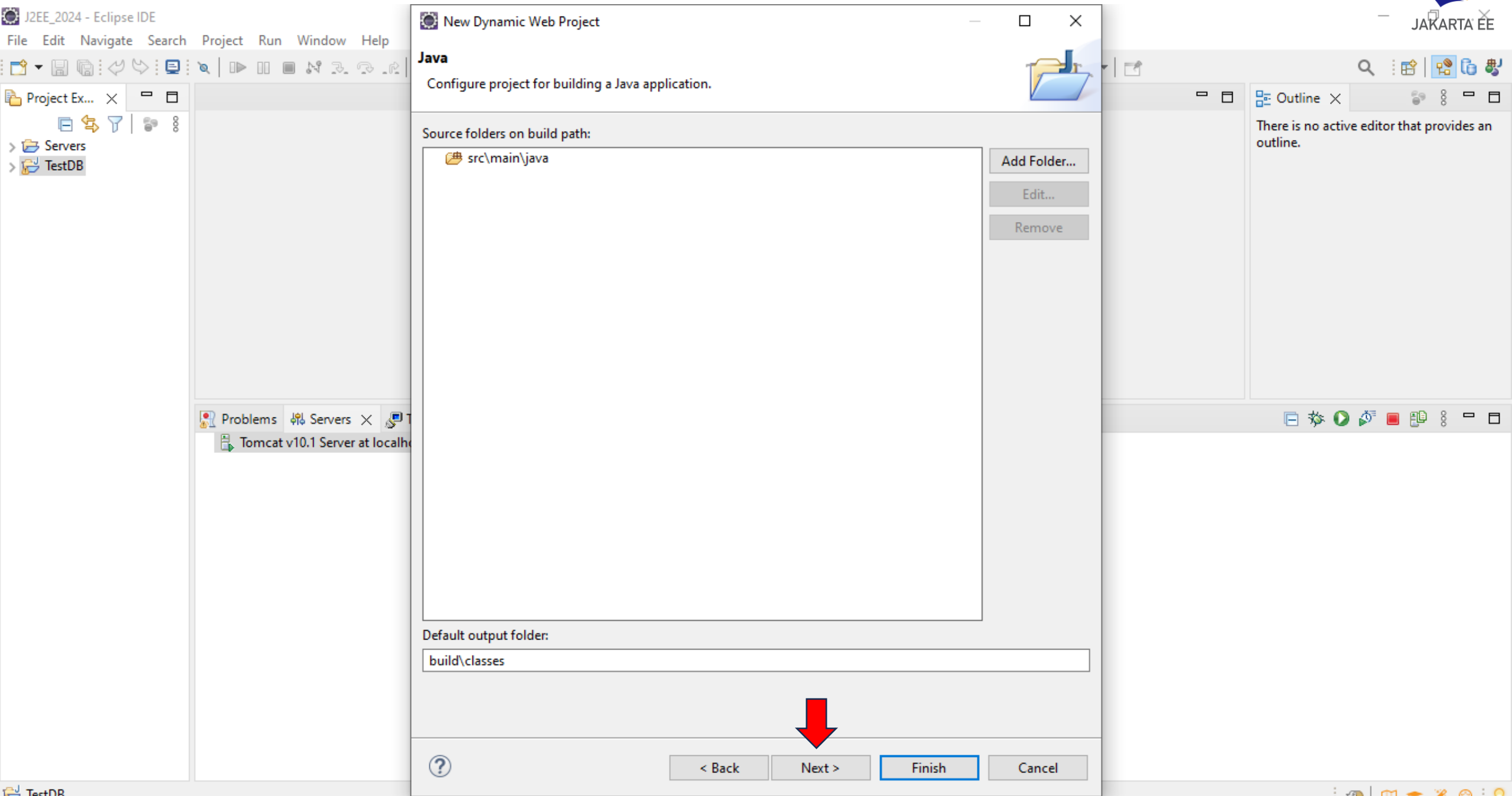
Creating a Web Application



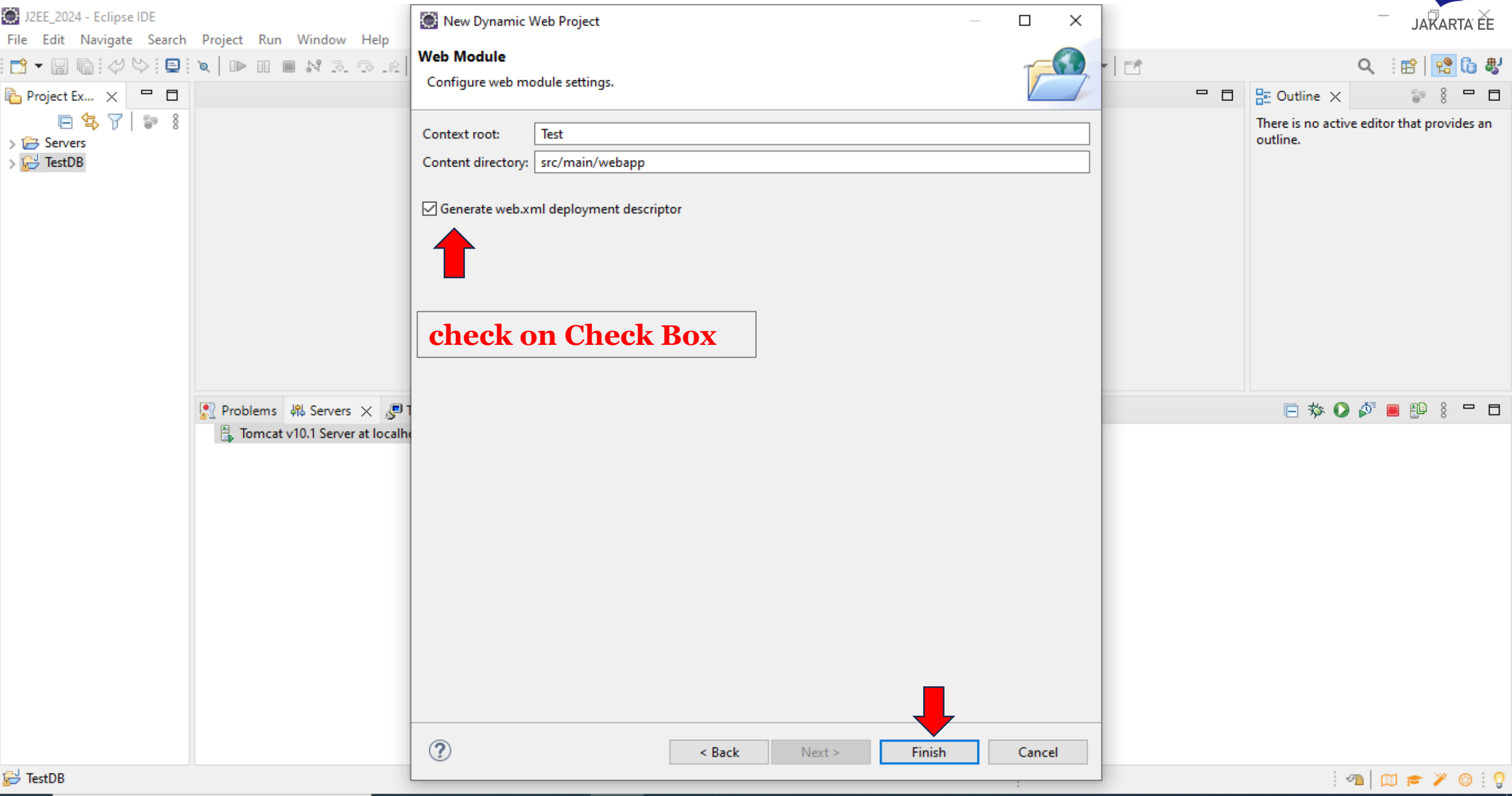
Creating a Web Application



Creating a Web Application



Creating a Web Application



REFERENCES

https://www.pragimtech.com/blog/blazor/what-are-restful-apis/

https://realpython.com/api-integration-in-python/